רשתות נוירונים רקורסיביות: מבוא מקיף

Recurrent Neural Networks: A Comprehensive Introduction

ד"ר יורם סגל

© Dr. Segal Yoram - כל הזכויות שמורות

2025

תוכן העניינים

רשימת איורים **5**

רשימת טבלאות **9**

1 למה רשתות RNN קיימות: Why RNNs Exist **11**

1.1.1 הבעיה של נתונים סדרתיים: The Sequential Data Problem 11

2 מבנה בסיסי של RNN: RNN Basic Structure **14**

2.2.1 רכיבים מרכזיים: Core Components 14

2.2.2 מבנה השכבות ב-RNN: Layer Architecture in RNN 16

2.2.3 משוב פנימי לכל פרספטרון: Internal Feedback per Perceptron 17

2.2.4 כיצד קלט אחד הופך לרבים: How One Input Becomes Many 19

2.2.5 פונקציות הפעלה ב-RNN: Activation Functions in RNN 21

2.2.6 פונקציות שגיאה (ssoL) ב-RNN: Loss Functions in RNN 25

3 פריסת רשתות RNN: Unrolling RNNs **28**

3.3.1 מושג הפריסה: The Unrolling Concept 28

3.3.2 דוגמת עיבוד - נתוני יומיים: Processing Example - 2-Day Data 29

3.3.3 הרחבה ליותר ימים: Scaling to More Days 31

4 שיתוף משקלים: Weight Sharing **33**

4.4.1 עקרון הפרמטרים המשותפים: Shared Parameters Principle 33

4.4.2 למה זה חשוב: Why This Matters 34

5 בעיית הגרדיאנט: The Gradient Problem **38**

5.5.1 גרדיאנטים מתפוצצים: Exploding Gradients 38

5.5.2 גרדיאנטים מתכווצים: Vanishing Gradients 39

5.5.3 הקשר להתפשטות לאחור: Connection to Backpropagation 41

5.5.4 תצוגה מקדימה של הפתרון: רשתות LSTM: Solution Preview: LSTM Networks 41

רשימת האיורים

1.1 רשת Feedforward מסורתית: Traditional Feedforward Network - המבנה מציג קלטים קבועים (3 קלטים תמיד) המחוברים לשכבה נסתרת ולפלט. הרשת אינה יכולה להתאים את מספר הקלטים באופן דינמי. 12

2.1 רשת RNN עם אורך משתנה: RNN with Variable Length - הרשת מכילה תא RNN אחד עם לולאת משוב (החץ הכחול). אותה רשת יכולה לעבד סדרות באורכים שונים: 5, 9, או 20 צעדי זמן. 13

3.1 השוואה חזותית: Visual Comparison - משמאל: רשת Feedforward עם קלטים קבועים שאינה יכולה להשתנות. מימין: רשת RNN עם לולאת משוב שמאפשרת עיבוד רצפים באורכים שונים. 14

1.2 רכיבי ליבה של RNN: Core Components of RNN - הרשת מכילה משקלים (W_1 ו-W_2), הטיה (B_1), פונקציית אקטיבציה (σ), ולולאת משוב (בצבע אדום) שמחזירה את הפלט כקלט לצעד הבא. 15

2.2 פרספטרון RNN בודד עם משוב עצמי: Single RNN Perceptron with Self-Feedback. הנוירון מקבל קלט חיצוני x_t וזיכרון עצמי h_{t-1}, ומחזיר את הפלט h_t אשר מוזן בחזרה לאותו נוירון בצעד הזמן הבא. 18

3.2 שכבה חבויה עם 4 נוירונים - כל אחד עם משוב עצמי: Hidden Layer with 4 Neurons - Each with Own Feedback. כל נוירון שומר את המצב החבוי שלו מהצעד הקודם ומזין אותו בחזרה לעצמו. המשוב הוא **פנימי** לכל נוירון - לא מועבר בין נוירונים שונים. 19

4.2 עיבוד רציף: Sequential Processing - שלושה צעדי זמן (t=1, t=2, t=3) שבהם כל תא RNN מקבל קלט ומעביר את המצב הסמוי (Y_1, Y_2) לצעד הבא. החצים הכחולים מראים את זרימת המידע בין הצעדים. 21

5.2 מנגנון לולאת המשוב: Feedback Loop Mechanism - פירוט של איך הפלט Y_t מתפצל לשני כיוונים: אחד כפלט סופי (ירוק) ואחד חוזר כקלט לצעד הבא (כחול). הנוסחה מראה כיצד Y_{t-1} מהצעד הקודם משולב עם $_t$Input הנוכחי. 22

6.2 השוואת פונקציות הפעלה: Activation Functions Comparison - ארבעה גרפים: Sigmoid (שמאל למעלה, כחול, טווח 0-1), Tanh (ימין למעלה, ירוק, טווח 1- עד 1, ממורכז באפס), ReLU (שמאל למטה, אדום, ללא גבול עליון), והשוואה ישירה של שלושתן (ימין למטה). כל פונקציה מתאימה לשימושים שונים ב-RNN. 24

7.2 תא RNN עם פונקציית הפעלה: RNN Cell with Activation Function - הקלט x_t והמצב הקודם h_{t-1} נכנסים לשכבה מחוברת במלואן (אדום), מחושב סכום משוקלל z, ואז עובר דרך פונקציית האקטיבציה σ (ירוק) לייצר את המצב החדש h_t. הנוסחה המלאה: $h_t = \sigma(W_1 \cdot x_t + W_2 \cdot h_{t-1} + b)$. 25

8.2 פונקציות שגיאה ב-RNN: Loss Functions in RNN - ארבעה דיאגרמות: MSE (משמאל למעלה) לרגרסיה, Cross-Entropy (מימין למעלה) לסיווג, שגיאה לאורך צעדי זמן (משמאל למטה), ודיאגרמת חישוב שגיאה ברצף (מימין למטה) שמראה איך השגיאה הכוללת היא סכום השגיאות מכל תא RNN. 27
1.3 רשת RNN מקופלת: Folded RNN - המצג הקומפקטי של רשת RNN עם לולאת המשוב (החץ הכחול העגול) שמחזירה את המצב הסמוי h כקלט לצעד הבא. מבנה מעגלי זה קשה לדמיון. 29
2.3 פריסה ל-2 צעדי זמן: Unrolled for 2 Time Steps - הרשת נפרשת לשני תאים: t=0 מעבד את מחיר אתמול (100) ו-t=1 מעבד את מחיר היום (105). המצב הסמוי h_0 זורם מהתא הראשון לשני (חץ כחול), ומאפשר לרשת לזהות את המגמה העולה. 30
3.3 פריסה ל-4 צעדי זמן: Unrolled for 4 Time Steps - רצף של ארבעה תאי RNN המעבדים מחירי מניות: 100, 105, 103, 107 שקלים. כל תא מעביר את המצב הסמוי שלו (h_0, h_1, h_2, h_3) לתא הבא (חצים כחולים). התא האחרון מייצר את החיזוי הסופי על בסיס כל ההיסטוריה. 32
4.3 השוואה: מקופל לעומת פרוס: Comparison: Folded vs Unrolled - משמאל: רשת RNN מקופלת עם לולאת משוב מעגלית. מימין: אותה רשת נפרשת ל-3 צעדי זמן, והלולאה הופכת לשרשרת ליניארית ברורה. החץ הכתום מראה את הטרנספורמציה הקונספטואלית. 33
1.4 שיתוף משקלים ברשת פרושה: Weight Sharing in Unrolled RNN - שלושה צעדי זמן (t-1, t, t+1) משתמשים באותם פרמטרים בדיוק: W_1 (כחול), W_2 (אדום), ו-B_1 (ירוק). החצים הכתומים המקווקוים מראים שכל התאים מחוברים לאותו מאגר פרמטרים משותף (התיבה הצהובה בתחתית). . . . 34
2.4 השוואת מספר פרמטרים: Parameter Count Comparison - משמאל: ללא שיתוף משקלים - כל צעד זמן דורש פרמטרים נפרדים (סה"כ 12 פרמטרים עבור 4 צעדים). מימין: עם שיתוף משקלים - רק 3 פרמטרים משותפים לכל הצעדים (החצים הירוקים המקווקוים). 35
3.4 יעילות זיכרון: Memory Efficiency - למעלה: 100 תאי RNN בפריסה ויזואלית (מיוצגים ב-20 תיבות כחולות). למטה: רק 3 פרמטרים (W_1, W_2, B_1) מאוחסנים בזיכרון פעם אחת. היעילות: זיכרון קבוע, אימון מהיר יותר, והכללה טובה יותר. 36
4.4 גרף התרחבות: Scaling Graph - הקו האדום מראה התפוצצות לינארית של מספר הפרמטרים ללא שיתוף משקלים (עד 300 פרמטרים עבור רצף באורך 100). הקו הירוק מראה שעם שיתוף משקלים, מספר הפרמטרים נשאר קבוע ב-3 ללא קשר לאורך הרצף! 37
1.5 התנהגות גרדיאנט מתפוצץ: Exploding Gradient Behavior - כאשר $W_2 = 2$, הגרדיאנט גדל באופן אקספוננציאלי ככל שאורך הסדרה גדל. הגרף מציג עקומת צמיחה אקספוננציאלית שמגיעה לערכים אסטרונומיים. 39

2.5 התנהגות גרדיאנט מתכווץ: Vanishing Gradient Behavior - כאשר $W_2 = 0.5$, הגרדיאנט קורס באופן אקספוננציאלי ככל שאורך הסדרה גדל. הגרף מציג עקומת ריקבון אקספוננציאלית שמתקרבת במהירות לאפס. 40
3.5 השוואת התנהגות גרדיאנטים: Gradient Behavior Comparison - השוואה בין גרדיאנט ב-RNN רגיל (קווים אדום וכחול מציגים התפוצצות והתכווצות) לבין גרדיאנט ב-LSTM (קו ירוק יציב). ניכר שה-LSTM שומר על גרדיאנט יציב גם בסדרות ארוכות. 42

© Dr. Segal Yoram - כל הזכויות שמורות

רשימת הטבלאות

1.1 השוואה בין רשתות מסורתיות לרשתות RNN: Comparison: Traditional vs RNN Networks 13
1.4 השוואת מספר הפרמטרים: Parameter Count Comparison 36
1.5 סיכום בעיות הגרדיאנט: Gradient Problems Summary 43

1 למה רשתות RNN קיימות: Why RNNs Exist

בעולם של Machine Learning ו-Deep Learning, רשתות נוירונים מסורתיות עובדות בצורה מצוינת עבור בעיות רבות. אולם, ישנו סוג מיוחד של נתונים שמציב אתגר ייחודי: נתונים סדרתיים או רציפים. פרק זה יסביר מדוע נתונים סדרתיים דורשים ארכיטקטורה מיוחדת של רשתות נוירונים, וכיצד רשתות RNN (skrowteN larueN tnerruceR - רשתות נוירונים רקורסיביות) פותרות בעיה זו.

1.1.1 הבעיה של נתונים סדרתיים: The Sequential Data Problem

נתונים רבים בעולם האמיתי הם סדרתיים במהותם - הם משתנים לאורך זמן ובעלי סדר חשיבות. בואו נבחן דוגמה קלאסית: מחירי מניות בבורסה.

דוגמה: ניבוי מחירי מניות

נניח שאנחנו רוצים לחזות את מחיר המניה של חברת TechCorp למחר. איזה מידע נשתמש? בוודאי נרצה להסתכל על מחירי המניה בימים האחרונים. הנה הבעיה המעניינת:

- **משקיע א'** בוחן את מחירי המניה מ-9 הימים האחרונים
- **משקיע ב'** בוחן רק את 5 הימים האחרונים
- **משקיע ג'** מעדיף להסתכל על 20 ימים לאחור

כל משקיע משתמש באורך שונה של היסטוריה. זו בדיוק הבעיה שרשתות נוירונים מסורתיות לא יכולות לפתור.

המגבלה של רשתות Feedforward מסורתיות

רשת נוירונים מסורתית (Feedforward Neural Network) בנויה עם מספר קבוע של קלטים. לדוגמה:

- אם נבנה רשת עם 9 קלטים - היא תוכל לקבל רק סדרות באורך 9
- משקיע ב' עם 5 ימים לא יוכל להשתמש ברשת שלנו (חסרים 4 ערכים)
- משקיע ג' עם 20 ימים גם לא יוכל להשתמש ברשת (יש 11 ערכים מיותרים)

זוהי **מגבלת הקלט הקבוע**. רשת מסורתית דורשת תמיד את אותו מספר קלטים בדיוק.

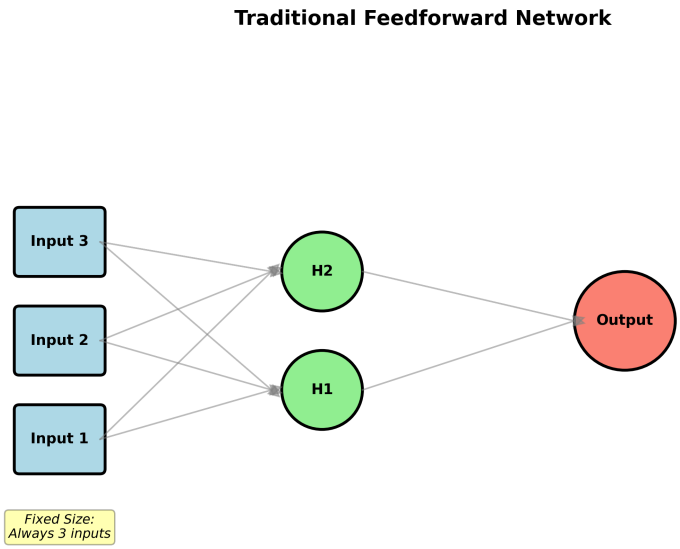
איור 1.1 מציג את המבנה של רשת Feedforward מסורתית. ניתן לראות בבירור שיש בדיוק 3 קלטים המחוברים לשכבה הנסתרת. כל קלט מחובר לכל נוירון בשכבה הנסתרת, והנוירונים הנסתרים מחוברים לפלט. זהו מבנה קבוע שלא ניתן לשנות - אם הנתונים שלנו מכילים 5 ערכים או 20 ערכים, הרשת לא תוכל לעבד אותם.

דוגמאות נוספות לנתונים סדרתיים

הבעיה הזו לא ייחודית למניות. היא מופיעה בתחומים רבים:

- **ניתוח טקסט**: משפטים באורכים שונים (מילה 1 עד מאות מילים)

© Dr. Segal Yoram - כל הזכויות שמורות



איור 1.1: רשת Feedforward מסורתית: Traditional Feedforward Network - המבנה מציג קלטים קבועים (3 קלטים תמיד) המחוברים לשכבה נסתרת ולפלט. הרשת אינה יכולה להתאים את מספר הקלטים באופן דינמי.

- **עיבוד אודיו**: קטעי דיבור באורכים משתנים
- **תחזית מזג אוויר**: נתוני טמפרטורה מספר שונה של ימים
- **ניטור רפואי**: מדידות של דופק או לחץ דם לאורך פרקי זמן משתנים
- **תנועה בסרטון**: רצפים של פריימים באורכים שונים

בכל אחד מהמקרים הללו, אורך הסדרה משתנה מדגימה לדגימה.

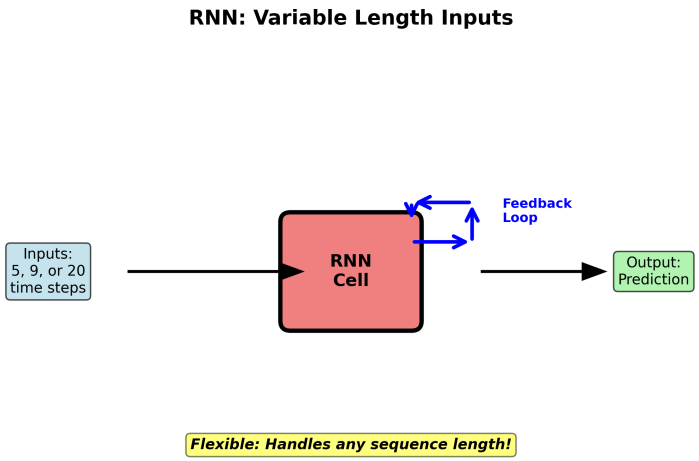
הצורך בגמישות

מה שאנחנו צריכים הוא רשת שיכולה:

1. **לעבד סדרות באורכים משתנים** - 5 ימים, 9 ימים, 20 ימים - כולם תקינים
2. **לשמור על מידע מהעבר** - לזכור מה קרה בצעדים קודמים
3. **לעדכן את הידע עם כל קלט חדש** - לשלב מידע חדש עם ידע קיים
4. **להפיק תחזית** - על בסיס כל ההיסטוריה שנצברה

זו בדיוק המטרה של רשתות RNN!

באיור 1.2 רואים את הפתרון המבריק של רשתות RNN. במקום מספר קבוע של קלטים, יש תא RNN אחד עם לולאת משוב (מסומנת בכחול). הלולאה הזו מאפשרת לרשת לעבד קלט אחד בכל פעם, לשמור את המידע במצב פנימי, ואז לעבד את הקלט הבא תוך שימוש במידע שנשמר. כך, אותה רשת יכולה לעבד 5 קלטים, 9 קלטים, או כל מספר אחר - פשוט על ידי הרצת הלולאה מספר פעמים שונה!



איור 2.1: רשת RNN עם אורך משתנה: RNN with Variable Length - הרשת מכילה תא RNN אחד עם לולאת משוב (החץ הכחול). אותה רשת יכולה לעבד סדרות באורכים שונים: 5, 9, או 20 צעדי זמן.

טבלה 1.1: השוואה בין רשתות מסורתיות לרשתות RNN: Comparison: Traditional vs RNN Networks

Criterion / קריטריון	**רשת Feedforward מסורתית**	**רשת RNN**
Input Length / אורך קלט	קבוע ומוגדר מראש	משתנה וגמיש
Example / דוגמה	תמיד 9 ערכים בדיוק	5, 9, 20 - כל אורך תקין
quential Handling / טיפול בסדרות	רואה את כל הקלטים בבת אחת	מעבד צעד אחר צעד
Memory / זיכרון	אין זיכרון פנימי	שומר מידע מצעדים קודמים
Flexibility / גמישות	נמוכה - דורש בניה מחדש	גבוהה - אותה רשת לכל אורך
Suitable for / מתאים ל-	תמונות, נתונים טבלאיים	טקסט, אודיו, סדרות זמן

טבלת השוואה: קלטים קבועים מול קלטים משתנים

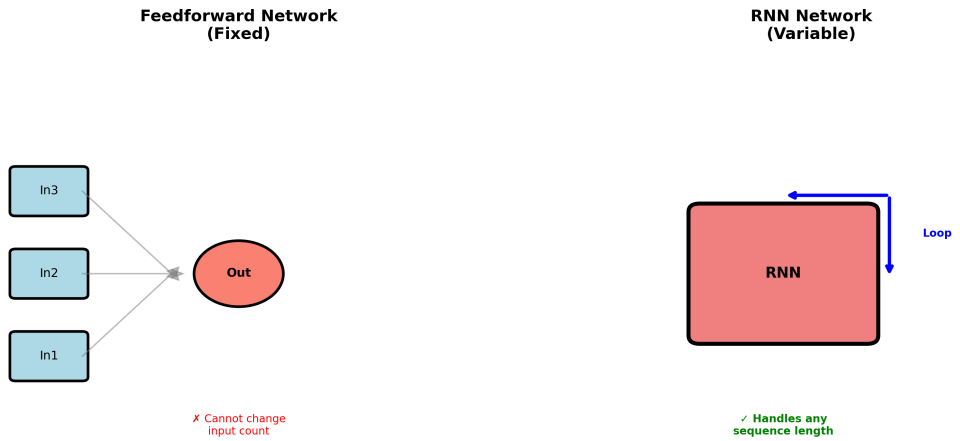
הטבלה 1.1 מסכמת את ההבדלים המרכזיים בין שני הגישות.

איור 5.3 מציג השוואה ויזואלית ברורה בין שתי הגישות. בצד שמאל רואים רשת Feedforward מסורתית עם 3 קלטים קבועים - אם ננסה להוסיף קלט רביעי, הרשת לא תדע מה לעשות איתו. בצד ימין רואים רשת RNN עם לולאת משוב (המעגל הכחול) - אותה רשת יכולה לעבד כל מספר קלטים פשוט על ידי הרצת הלולאה מספר פעמים מתאים.

סיכום הפרק

הבנו שנתונים סדרתיים מציבים שלושה אתגרים מרכזיים:

1. **אורכים משתנים** - סדרות שונות באורכים שונים
2. **תלות בזמן** - הסדר של הערכים חשוב (אתמול לפני היום)
3. **הקשר ארוך טווח** - מה שקרה לפני 10 צעדים עשוי להיות רלוונטי



איור 3.1: השוואה חזותית: Visual Comparison - משמאל: רשת Feedforward עם קלטים קבועים שאינה יכולה להשתנות. מימין: רשת RNN עם לולאת משוב שמאפשרת עיבוד רצפים באורכים שונים.

רשתות נוירונים מסורתיות לא מסוגלות להתמודד עם אתגרים אלה בגלל המבנה הקבוע שלהן. זו הסיבה שרשתות RNN הומצאו - כדי לספק את הגמישות והזיכרון הדרושים לעיבוד נתונים סדרתיים.

בפרקים הבאים נלמד כיצד רשתות RNN פועלות, מה המבנה הפנימי שלהן, וכיצד הן משיגות את היכולות המיוחדות האלה.

2 מבנה בסיסי של RNN: RNN Basic Structure

הבנת המבנה הבסיסי של רשתות RNN היא המפתח להבנת יכולותיהן. בפרק זה נבחן את הרכיבים המרכזיים של הרשת ונראה כיצד לולאת המשוב מאפשרת לרשת לעבד נתונים רציפים. מהי ההבדל המהותי בין רשת נוירונים רגילה לבין RNN? התשובה טמונה בלולאת המשוב שמאפשרת לרשת "לזכור" מידע מצעדי זמן קודמים.

2.2.1 רכיבים מרכזיים: Core Components

רשתות RNN מורכבות מאותם אבני בניין בסיסיים שאנו מכירים מרשתות נוירונים רגילות, אך עם הוספה קריטית אחת - לולאת המשוב.

משקלים והטיות: כמו בכל רשת נוירונים, גם ברשתות RNN יש משקלים (Weights) והטיות (Biases). המשקלים קובעים את עוצמת הקשרים בין הנוירונים, וההטיות מאפשרות לרשת להתאים את רמת ההפעלה הבסיסית. בהקשר של RNN, אנו מבחינים בין מספר סוגי משקלים:

1. W_1 - משקל הקלט, שמחבר את הערך הנוכחי בסדרה לנוירון
2. W_2 - משקל המשוב, שמחבר את הפלט הקודם חזרה לרשת

כל הזכויות שמורות - Dr. Segal Yoram ©

3. B_1 - הטיה, שמתווספת לסכום המשוקלל

פונקציות הפעלה: פונקציות ההפעלה (Activation Functions) ברשתות RNN פועלות בדיוק כמו ברשתות רגילות. הן מוסיפות אי־לינאריות לרשת ומאפשרות לה ללמוד תבניות מורכבות. פונקציות נפוצות כוללות tanh, sigmoid, ו־ReLU. בדוגמאות שלנו נשתמש בפונקציית הפעלה כללית שמסומנת כ־activation$(\cdot)$.

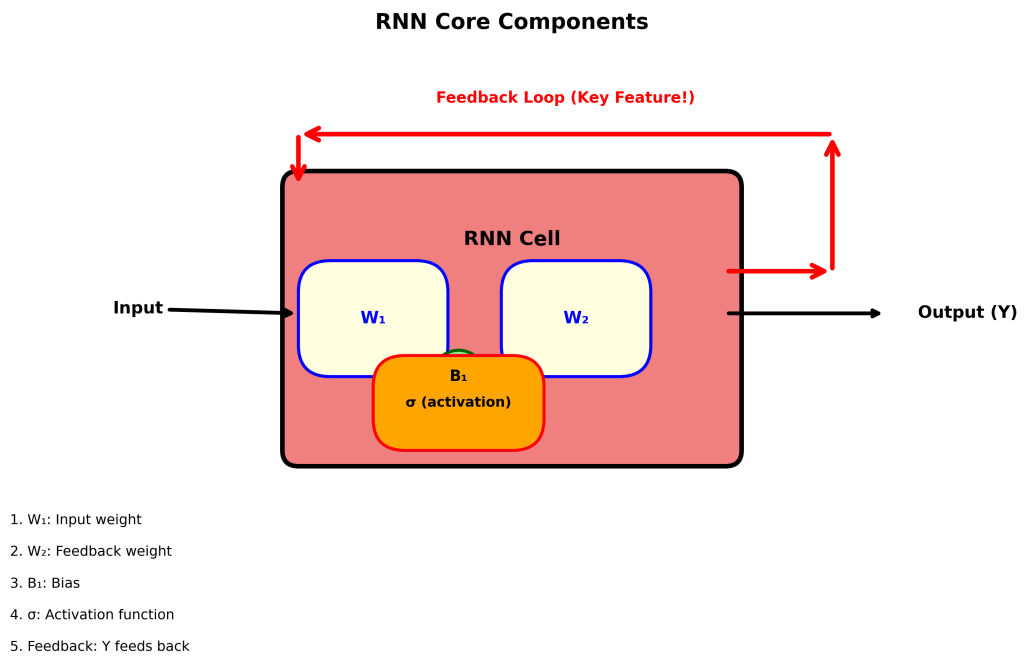
לולאת המשוב - ההבדל המהותי: זהו האלמנט המבדל את רשתות RNN מרשתות Feed-forward רגילות. לולאת המשוב (Feedback Loop) מאפשרת לפלט של הרשת בצעד זמן אחד להשפיע על החישוב בצעד הזמן הבא. למעשה, הרשת "זוכרת" מה עיבדה קודם ומשתמשת במידע זה כדי לעבד את הקלט הנוכחי.

כיצד זה עובד? בואו נראה את הנוסחה הבסיסית לצעד בודד ברשת RNN:

$$Y_1 = \text{activation}(W_1 \times \text{input} + B_1) \tag{2.1}$$

משוואה 2.1 מתארת חישוב פשוט: אנו לוקחים את הקלט, מכפילים אותו במשקל W_1, מוסיפים הטיה B_1, ומפעילים פונקציית הפעלה. עד כאן, זה נראה בדיוק כמו רשת נוירונים רגילה.

אבל מה קורה עם הפלט Y_1? כאן מתחיל הקסם של RNN.



איור 1.2: רכיבי ליבה של RNN: Core Components of RNN - הרשת מכילה משקלים (W_1 ו-W_2), הטיה (B_1), פונקציית אקטיבציה (σ), ולולאת משוב (בצבע אדום) שמחזירה את הפלט כקלט לצעד הבא.

© Dr. Segal Yoram - כל הזכויות שמורות

איור 2.1 מציג את כל הרכיבים המרכזיים של תא RNN. ניתן לראות את המשקל W_1 שמעבד את הקלט הנוכחי, המשקל W_2 שמעבד את המשוב מהצעד הקודם, ההטיה B_1, ופונקציית האקטיבציה σ. החץ האדום הגדול מראה את לולאת המשוב - זהו המרכיב המבדל בין RNN לרשת רגילה.

2.2.2 מבנה השכבות ב-RNN: Layer Architecture in RNN

נקודה חשובה - שכבות מחוברות במלואן

רשתות RNN בנויות על בסיס שכבות **מחוברות במלואן** (Fully Connected Layers) או MLP (Multi-Layer Perceptron). כלומר, כל נוירון בשכבה מחובר לכל נוירון בשכבה הבאה. זהו מבנה שונה לחלוטין מסוגי ארכיטקטורות אחרות שאתם אולי מכירים.

המבנה הבסיסי: רשת RNN סטנדרטית מכילה שלוש רכיבים מרכזיים:

1. **שכבת קלט** (Input Layer) - מקבלת את הטוקן הנוכחי בכל צעד זמן
2. **שכבה חבויה חוזרת** (Recurrent Hidden Layer) - לפחות שכבה אחת, שמחשבת את המצב הסמוי (Hidden State)
3. **שכבת פלט** (Output Layer) - מייצרת את התוצאה לאותו צעד זמן

השכבה החבויה - הלב של ה-RNN: השכבה החבויה היא המרכיב המרכזי שמבצע את **החיבור החוזר**. בכל צעד זמן, השכבה החבויה:

- מקבלת קלט מהשכבת הקלט (הטוקן הנוכחי)
- מקבלת את המצב הסמוי מהצעד הקודם (הזיכרון)
- מחשבת מצב סמוי חדש על ידי שילוב שני המקורות
- מעבירה את המצב החדש לשכבת הפלט ולצעד הזמן הבא

כל הקשרים בשכבה החבויה הם **קשרים מלאים** - אין קשרים חלקיים או מקומיים כמו בארכיטקטורות אחרות.

רשתות RNN עמוקות: רשת RNN קלאסית בדרך כלל מורכבת משכבה חבויה אחת. אולם, ניתן להעמיק את הרשת על ידי הוספת עוד שכבות חבויות:

- **רשת RNN רדודה:** שכבת קלט $\rightarrow$ שכבה חבויה אחת $\rightarrow$ שכבת פלט
- **רשת RNN עמוקה:** שכבת קלט $\rightarrow$ שכבה חבויה 1 $\rightarrow$ שכבה חבויה 2 $\rightarrow$ ... $\rightarrow$ שכבת פלט

חשוב להדגיש: כל הוספת עומק היא **עוד שכבה חבויה מלאה** (Fully Connected) עם קשרים חוזרים, ולא סוג אחר של שכבה.

כל הזכויות שמורות - Dr. Segal Yoram ©

טוקנים בכניסה לרשת: בכניסה לרשת RNN בדרך כלל מזינים **טוקנים** (Tokens) - יחידות בסיסיות של נתונים:

- **בעיבוד שפה טבעית** (NLP): כל מילה או תו הוא טוקן
- **בסדרות זמן**: כל מדידה (למשל, מחיר מניה ביום מסוים) היא טוקן
- **באודיו**: כל מקטע קצר של אות הוא טוקן

הטוקנים נוצרים בתהליך שנקרא **טוקניזציה** (Tokenization), בו מחלקים את רצף הנתונים ליחידות בסיסיות. תהליך זה מתבצע על ידי ספריות עיבוד נתונים כמו NLTK, SpaCy, או רכיבים מובנים ב-Transformers.

לאחר הטוקניזציה, הטוקנים עוברים בדרך כלל **הטמעה** (Embedding) - כל טוקן הופך לווקטור צפוף של מספרים שמייצג אותו במרחב רב-ממדי.

סיכום מבנה השכבות:

- רשתות RNN בנויות משכבות מחוברות במלואן (Fully Connected)
- כל נוירון בשכבה מחובר לכל נוירון בשכבה הבאה
- השכבה החבויה היא הרכיב המרכזי שמבצע את החיבור החוזר
- הקלטים הם טוקנים שעברו תהליך טוקניזציה והטמעה
- ניתן להעמיק את הרשת על ידי הוספת שכבות חבויות נוספות

2.2.3 משוב פנימי לכל פרספטרון: Internal Feedback per Perceptron

נקודה קריטית - זיכרון ומשוב ברמת הנוירון הבודד:

ברשת RNN, המשוב ("Hidden State" או "מצב חבוי") תמיד נשמר ומוזן **ברמת הפרספטרון הבודד**. כלומר:

- כל נוירון בשכבה החבויה שומר את הערך הקודם **שלו עצמו**
- המשוב הוא **פנימי** לכל פרספטרון - לא מועבר בין נוירונים שונים
- אם יש לנו n נוירונים בשכבה החבויה, כל אחד מהם שומר את ה-Hidden State שלו בלבד

פרספטרון בודד עם משוב עצמי: איור 2.2 מציג את המבנה של פרספטרון RNN בודד עם משוב עצמי. הנוירון מקבל שני קלטים:

1. **קלט חיצוני** x_t: הקלט הנוכחי בצעד הזמן t
2. **זיכרון עצמי** h_{t-1}: הפלט של **אותו נוירון** מהצעד הקודם

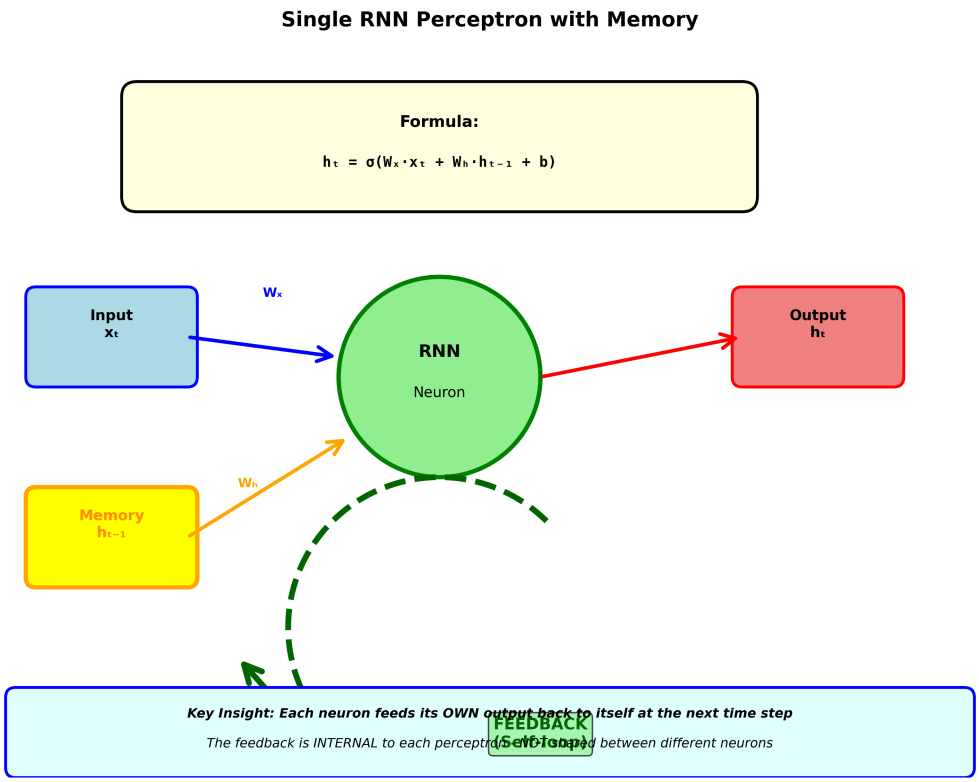
© Dr. Segal Yoram - כל הזכויות שמורות

הנוירון מחשב את הפלט החדש h_t לפי הנוסחה:

$$h_t = \sigma(W_x \cdot x_t + W_h \cdot h_{t-1} + b) \tag{2.2}$$

כאשר:

- W_x - משקל הקלט החיצוני
- W_h - משקל המשוב העצמי (Recurrent Weight)
- b - ביאס
- σ - פונקציית הפעלה (בדרך כלל Tanh)



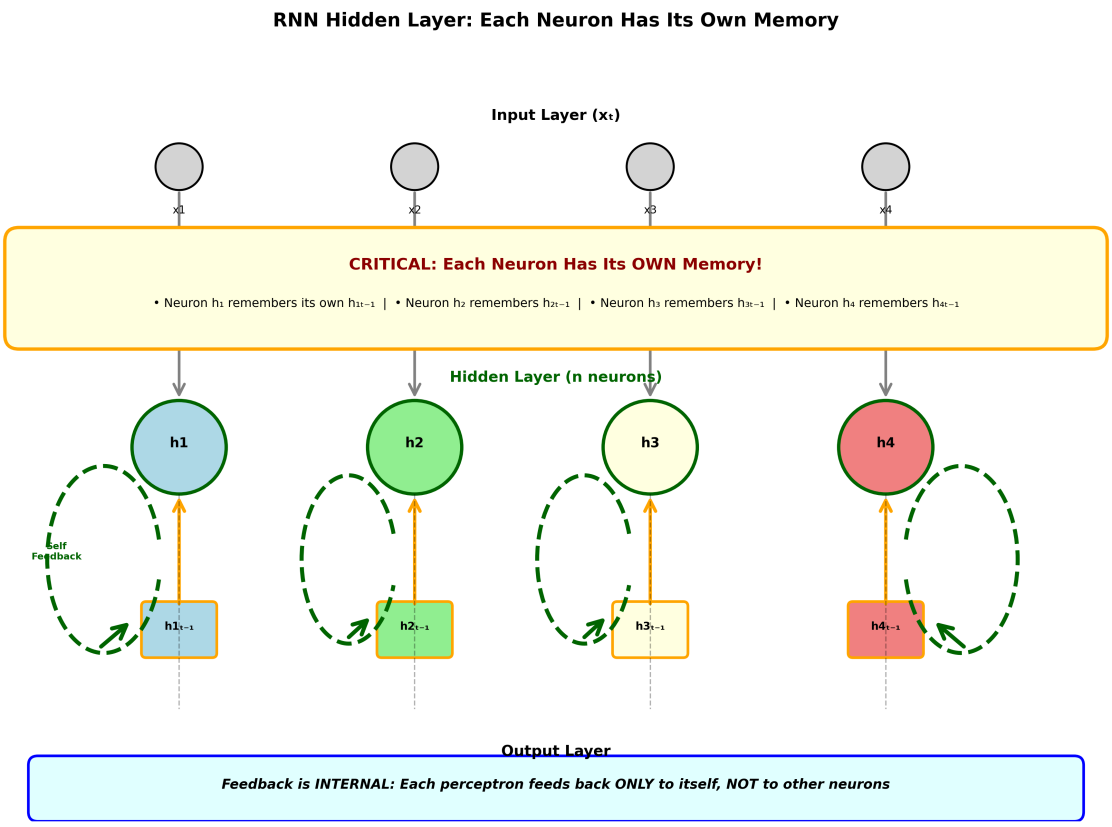
איור 2.2: פרספטרון RNN בודד עם משוב עצמי: Single RNN Perceptron with Self-Feedback. הנוירון מקבל קלט חיצוני x_t וזיכרון עצמי h_{t-1}, ומחזיר את הפלט h_t אשר מוזן בחזרה לאותו נוירון בצעד הזמן הבא.

הלולאה החוזרת (המשוב) מסומנת בקו ירוק מקווקו. זו הלולאה שיוצרת את ה"Recurrence" ב-RNN - הנוירון מזין את עצמו בחזרה.

שכבה חבויה עם מספר נוירונים - כל אחד עם משוב משלו: איור 2.3 מציג שכבה חבויה המכילה 4 נוירונים. **כל נוירון שומר ומזין את המצב החבוי שלו בלבד:**

- נוירון h_1 זוכר את h_1^{t-1} שלו

- נוירון h_2 זוכר את h_2^{t-1} שלו
- נוירון h_3 זוכר את h_3^{t-1} שלו
- נוירון h_4 זוכר את h_4^{t-1} שלו



איור 3.2: שכבה חבויה עם 4 נוירונים - כל אחד עם משוב עצמי: Hidden Layer with 4 Neurons - Each with Own Feedback. כל נוירון שומר את המצב החבוי שלו מהצעד הקודם ומזין אותו בחזרה לעצמו. המשוב הוא **פנימי** לכל נוירון - לא מועבר בין נוירונים שונים.

הבנה מרכזית: המשוב אינו משותף בין הנוירונים. כל נוירון מתנהג כמו "יחידת זיכרון עצמאית" עם לולאת משוב פנימית משלו. זה מאפשר לכל נוירון ללמוד ולזכור תבניות שונות מהסדרה הזמנית.

כאשר כל הנוירונים בשכבה החבויה פועלים יחד, הם יוצרים **ייצוג מבוזר של הזיכרון** - כל נוירון תורם חלק מהמידע על ההיסטוריה של הסדרה.

2.2.4 כיצד קלט אחד הופך לרבים: How One Input Becomes Many

המפתח להבנת רשתות RNN הוא להבין שהפלט Y_1 לא פשוט יוצא מהרשת ונעלם. במקום זה, הוא מתפצל לשני כיוונים:

1. **כיוון אחד:** הפלט Y_1 ממשיך קדימה כפלט של הרשת

2. **כיוון שני:** הפלט Y_1 חוזר בלולאה ומשמש כקלט נוסף לצעד הבא

תהליך זה יוצר מנגנון זיכרון. כאשר הרשת מקבלת את הערך הבא בסדרה, היא לא מעבדת אותו בבידוד - היא גם מביאה בחשבון את מה שעיבדה בעבר!
הנוסחה המלאה לצעד הבא בסדרה היא:

$$\text{Sum} = (Y_1 \times W_2) + (\text{current_input} \times W_1) \tag{2.3}$$

משוואה 2.3 מראה את המהות של הקשר הרקורסיבי. הסכום המשוקלל מורכב משני חלקים:

- החלק הראשון: $Y_1 \times W_2$ - הפלט הקודם מוכפל במשקל המשוב
- החלק השני: $\text{current_input} \times W_1$ - הקלט הנוכחי מוכפל במשקל הקלט

עיבוד רציף של ערכים: בואו נעקוב אחר תהליך העיבוד לאורך מספר צעדי זמן:

צעד זמן 1:

- קלט: הערך הראשון בסדרה
- עיבוד: $Y_1 = \text{activation}(W_1 \times \text{input}_1 + B_1)$
- פלט: Y_1 (גם נשמר עבור הצעד הבא)

צעד זמן 2:

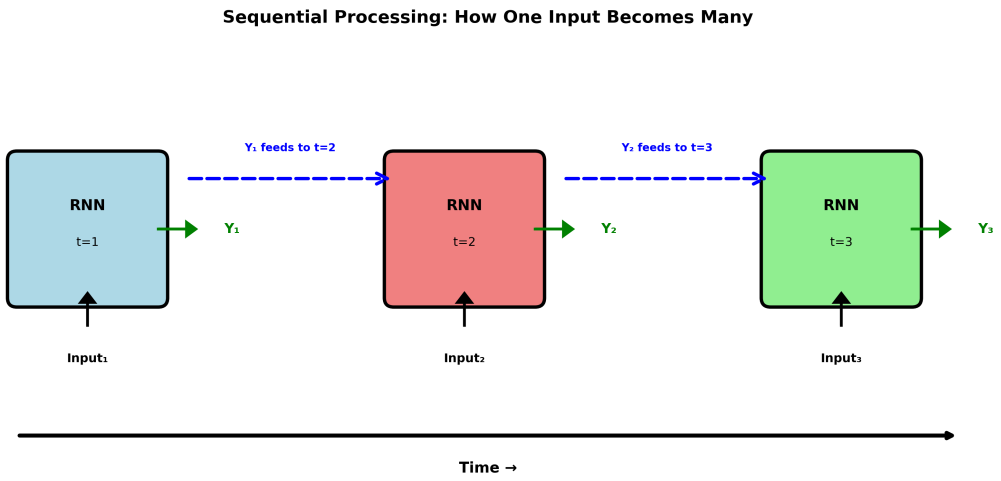
- קלט: הערך השני בסדרה + Y_1 מהצעד הקודם
- עיבוד: $\text{Sum}_2 = (Y_1 \times W_2) + (\text{input}_2 \times W_1)$
- המשך: $Y_2 = \text{activation}(\text{Sum}_2 + B_1)$
- פלט: Y_2 (גם נשמר עבור הצעד הבא)

צעד זמן 3:

- קלט: הערך השלישי בסדרה + Y_2 מהצעד הקודם
- עיבוד: $\text{Sum}_3 = (Y_2 \times W_2) + (\text{input}_3 \times W_1)$
- המשך: $Y_3 = \text{activation}(\text{Sum}_3 + B_1)$
- פלט: Y_3 (וכן הלאה...)

תהליך זה חוזר על עצמו לכל ערך בסדרה. כל פלט "זוכר" את כל הערכים שבאו לפניו, כיוון שהוא מושפע מהפלטים הקודמים דרך לולאת המשוב.
באיור 2.4 רואים את התהליך המלא של עיבוד רציף. בצעד t=1, הרשת מעבדת את הקלט הראשון ומייצרת Y_1. פלט זה מועבר לצעד t=2 (החץ הכחול המקווקו), שם הוא משולב עם הקלט השני. התהליך ממשיך לצעד t=3. שימו לב שכל צעד זמן מיוצג בצבע שונה, אך כולם משתמשים באותה רשת RNN!

כל הזכויות שמורות - Dr. Segal Yoram ©



איור 4.2: עיבוד רציף: Sequential Processing - שלושה צעדי זמן (t=1, t=2, t=3) שבהם כל תא RNN מקבל קלט ומעביר את המצב הסמוי (Y_1, Y_2) לצעד הבא. החצים הכחולים מראים את זרימת המידע בין הצעדים.

הכוח של לולאת המשוב: מנגנון פשוט זה מאפשר לרשת RNN לבצע דברים מדהימים:

1. **זיכרון:** הרשת יכולה "לזכור" דפוסים מצעדי זמן קודמים
2. **התאמה לאורך משתנה:** אותה רשת יכולה לעבד סדרות באורכים שונים
3. **הקשר:** הרשת מבינה שערכים בסדרה קשורים זה לזה ולא בלתי תלויים

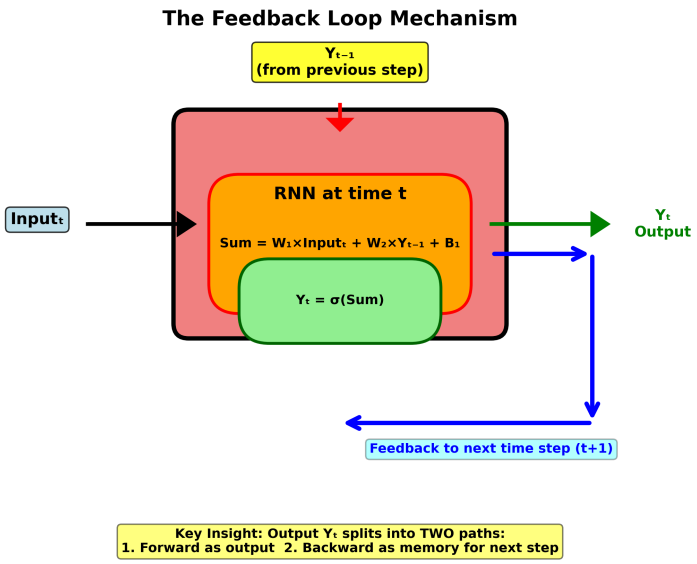
לדוגמה, אם הרשת מעבדת מחירי מניות משבוע שלם, הפלט לכל יום לא נקבע רק על סמך מחיר היום הנוכחי, אלא גם על סמך הטרנד שנצפה בימים הקודמים. זו בדיוק היכולת שחסרה לרשתות Feedforward רגילות.

איור 2.5 מפרט את המנגנון של לולאת המשוב. הקלט הנוכחי ($_t$Input) והפלט הקודם (Y_{t-1}, מסומן בצהוב) נכנסים ביחד לרשת. הם משולבים בנוסחה: $W_1 \times \texttt{Input}_t + = \texttt{muS}$ $W_2 \times Y_{t-1} + B_1$, ואז עוברים דרך פונקציית האקטיבציה. הפלט Y_t מתפצל: החץ הירוק הוא הפלט שיוצא מהרשת, והחץ הכחול מראה איך הוא חוזר בלולאה לצעד t+1.

נקודה חשובה - שיתוף פרמטרים: חשוב להדגיש שהמשקלים W_1, W_2 וההטיה B_1 **זהים** בכל צעדי הזמן. אנחנו לא לומדים משקולות שונות לכל צעד זמן - אנחנו משתמשים באותם משקולות שוב ושוב. עיקרון זה נקרא "שיתוף משקלים" (Weight Sharing) ונדון בו בהרחבה בפרק 4.

2.2.5 פונקציות הפעלה ב-RNN: Activation Functions in RNN

פונקציות ההפעלה הן רכיב קריטי ברשתות RNN. הן מוסיפות **אי-לינאריות** לרשת, ומאפשרות לה ללמוד תבניות מורכבות. בלי פונקציות הפעלה, הרשת הייתה רק שילוב לינארי של קלטים, ולא הייתה יכולה ללמוד יחסים מורכבים.



איור 5.2: מנגנון לולאת המשוב: Feedback Loop Mechanism - פירוט של איך הפלט Y_t מתפצל לשני כיוונים: אחד כפלט סופי (ירוק) ואחד חוזר כקלט לצעד הבא (כחול). הנוסחה מראה כיצד Y_{t-1} מהצעד הקודם משולב עם Input_t הנוכחי.

למה אנחנו צריכים אי-לינאריות? נניח שיש לנו שתי שכבות ברשת, ללא פונקציות הפעלה:

$$h_1 = W_1 x \tag{2.4}$$

$$h_2 = W_2 h_1 = W_2(W_1 x) = (W_2 W_1)x \tag{2.5}$$

התוצאה היא פשוט כפל של שתי מטריצות, שהוא עדיין טרנספורמציה לינארית! כל רשת עמוקה ללא פונקציות הפעלה שקולה לשכבה ליניארית אחת. פונקציות ההפעלה שוברות את הלינאריות ומאפשרות לרשת ללמוד פונקציות מורכבות.

פונקציות הפעלה נפוצות ב-RNN: 1. פונקציית Tanh (טנגנס היפרבולי):

זוהי הפונקציה הנפוצה ביותר ב-RNN מסורתיות. הנוסחה שלה:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{e^{2x} - 1}{e^{2x} + 1} \tag{2.6}$$

מאפיינים:

- **טווח:** $(-1, 1)$ - הפלט תמיד בין 1- ל-1
- **ממורכז באפס:** הפלט יכול להיות חיובי או שלילי
- **חלקה ורציפה:** מאפשרת גרדיאנטים יציבים

- **נגזרת:** $\tanh'(x) = 1 - \tanh^2(x)$

למה Tanh טובה ל-RNN? המרכוז באפס עוזר ללמידה - ערכים חיוביים ושליליים מאפשרים לרשת ללמוד כיוונים (עליה/ירידה במגמות).

2. פונקציית Sigmoid (סיגמואיד):

הנוסחה:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{2.7}$$

מאפיינים:

- **טווח:** $(0, 1)$ - הפלט תמיד חיובי
- **מתאימה להסתברויות:** שימושית בשכבת פלט לסיווג בינארי
- **בעיה:** לא ממורכזת באפס, גורמת ל-Vanishing Gradient
- **נגזרת:** $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

3. פונקציית ReLU (tinU raeniL deifitceR):

הנוסחה:

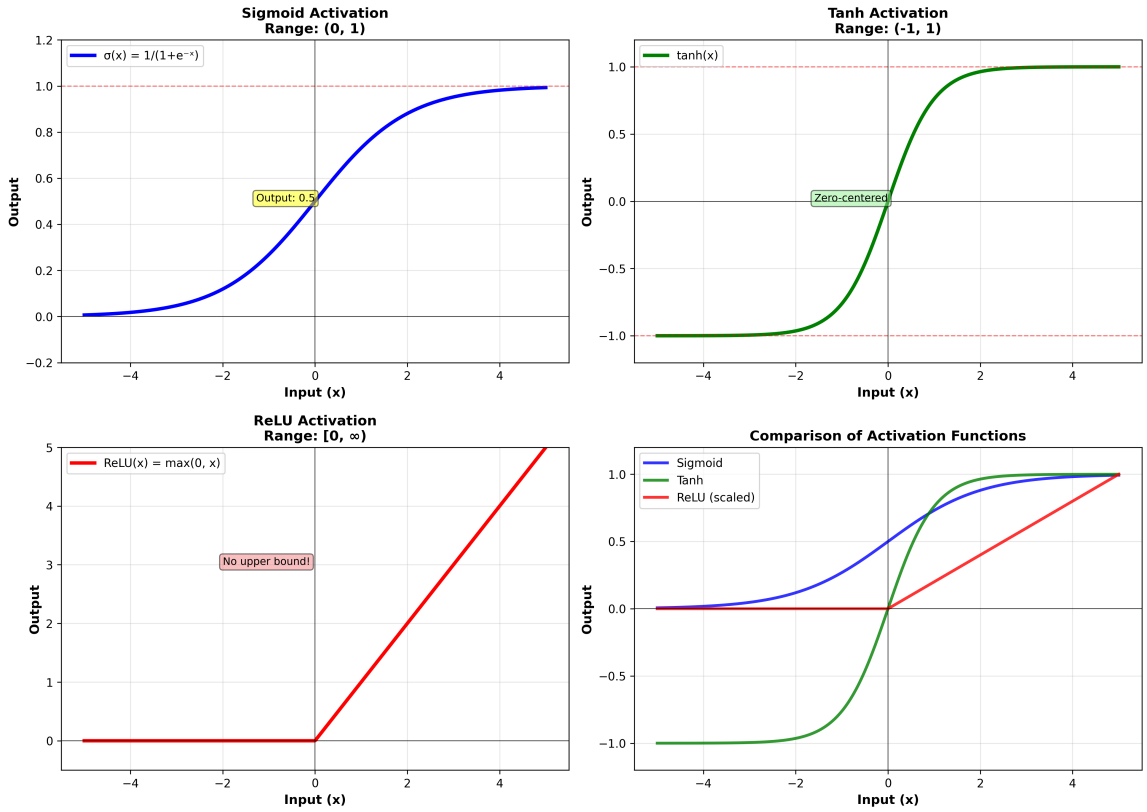
$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{2.8}$$

מאפיינים:

- **טווח:** $[0, \infty)$ - אין גבול עליון
- **פשוטה לחישוב:** מהירה מאוד
- **פותרת Vanishing Gradient:** הנגזרת היא 0 או 1
- **בעיה ב-RNN:** יכולה לגרום ל-Exploding Gradient בחיבורים חוזרים
- **נגזרת:** $\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$

השוואה ויזואלית של הפונקציות: איור 2.6 משווה ויזואלית את שלוש פונקציות ההפעלה המרכזיות ב-RNN. בגרף השמאלי העליון רואים את **Sigmoid** (בכחול) - פונקציה חלקה שמוגבלת לטווח 0 עד 1, שימושית להסתברויות אך סובלת מ-Vanishing Gradient. בגרף הימני העליון רואים את **Tanh** (בירוק) - דומה ל-Sigmoid אך ממורכזת באפס בטווח 1- עד 1, זוהי הבחירה המועדפת ב-RNN מסורתיות. בגרף השמאלי התחתון רואים את **ReLU** (באדום) - פונקציה פשוטה שמחזירה אפס לערכים שליליים והערך עצמו לערכים חיוביים, ללא גבול עליון. הגרף הימני התחתון משווה את שלושתן יחד - ניתן לראות בבירור ש-Tanh ו-Sigmoid דומות בצורה אך שונות בטווח, ואילו ReLU ליניארית לחלוטין עבור ערכים חיוביים.

© Dr. Segal Yoram - כל הזכויות שמורות



איור 6.2: השוואת פונקציות הפעלה: Activation Functions Comparison - ארבעה גרפים: Sigmoid (שמאל למעלה, כחול, טווח 0-1), Tanh (ימין למעלה, ירוק, טווח 1- עד 1, ממורכז באפס), ReLU (שמאל למטה, אדום, ללא גבול עליון), והשוואה ישירה של שלושתן (ימין למטה). כל פונקציה מתאימה לשימושים שונים ב-RNN.

איך פונקציית ההפעלה משתלבת ב-RNN: בכל צעד זמן, הרשת מבצעת שני שלבים:

שלב 1 - חישוב סכום משוקלל:

$$z_t = W_1 x_t + W_2 h_{t-1} + b \tag{2.9}$$

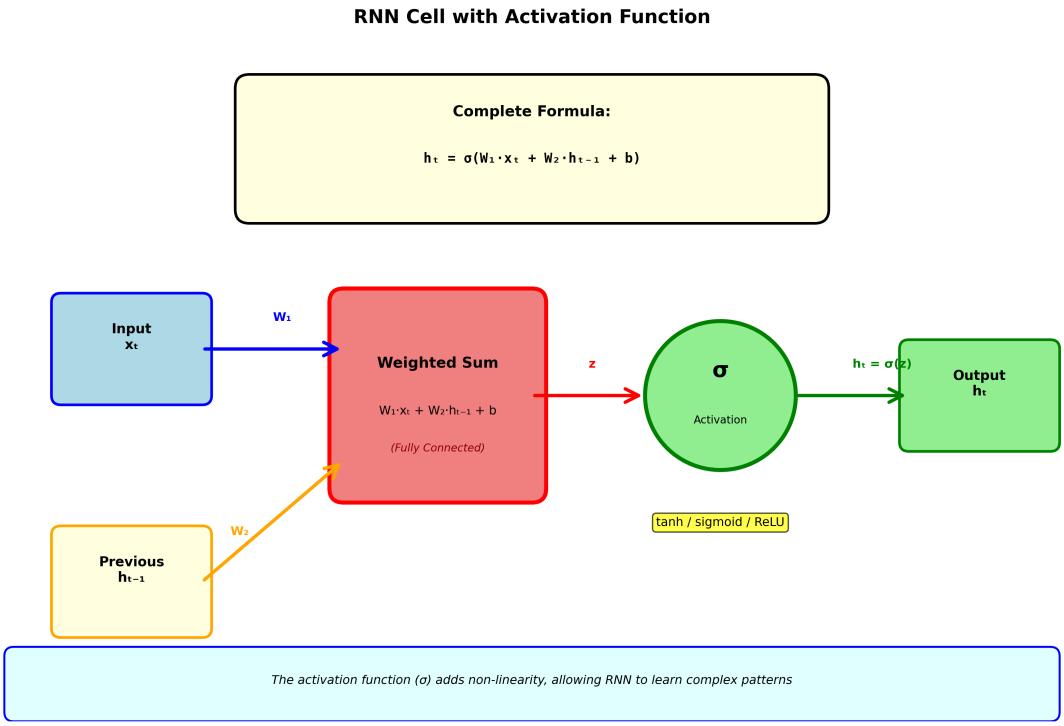
זהו חישוב לינארי - פשוט סכום של קלטים משוקללים.

שלב 2 - הפעלת פונקציית ההפעלה:

$$h_t = \sigma(z_t) \tag{2.10}$$

כאן σ יכולה להיות tanh, sigmoid, או ReLU. פונקציה זו הופכת את הסכום הלינארי לפלט לא-לינארי.

איור 2.7 מפרט את התהליך המלא בתא RNN. הקלט הנוכחי (כחול) והמצב הסמוי הקודם (צהוב) נכנסים לתיבה האדומה - השכבה המחוברת במלואן - שמחשבת סכום משוקלל. התוצאה z עוברת דרך העיגול הירוק (פונקציית האקטיבציה σ) ומייצרת את המצב הסמוי החדש h_t. ללא פונקציית האקטיבציה, הרשת לא הייתה יכולה ללמוד דפוסים מורכבים!



איור 7.2: תא RNN עם פונקציית הפעלה: RNN Cell with Activation Function - הקלט x_t והמצב הקודם h_{t-1} נכנסים לשכבה מחוברת במלואן (אדום), מחושב סכום משוקלל z, ואז עובר דרך פונקציית האקטיבציה σ (ירוק) לייצר את המצב החדש h_t. הנוסחה המלאה: $h_t = \sigma(W_1 \cdot x_t + W_2 \cdot h_{t-1} + b)$.

למה Tanh היא הבחירה הנפוצה? ברשתות RNN מסורתיות, tanh היא הבחירה המועדפת מסיבות אלו:

1. **ממורכזת באפס:** עוזרת ללמידה יציבה יותר
2. **טווח מוגבל:** מונעת ערכים קיצוניים
3. **גרדיאנטים סבירים:** הנגזרת לא קטנה מדי (בניגוד ל-sigmoid)

2.2.6 פונקציות שגיאה (ssoL) ב-RNN: Loss Functions in RNN

פונקציית השגיאה (או פונקציית ה-Loss) מודדת כמה הרשת טועה בחיזויים שלה. המטרה של האימון היא **למזער** את פונקציית השגיאה.

פונקציות שגיאה נפוצות: 1. שגיאת ריבוע ממוצעת - Mean Squared Error (MSE):

משמשת בבעיות **רגרסיה** (חיזוי ערכים רציפים):

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{2.11}$$

כאשר:

- y_i - הערך האמיתי (היעד)

- $\hat{y}_i$ - החיזוי של הרשת

- n - מספר הדוגמאות

דוגמה: חיזוי מחיר מניה למחר. אם המחיר האמיתי הוא 105 ש"ח והרשת חזתה 103 ש"ח, השגיאה היא: $(105-103)^2=4$.

2. אנטרופיה צולבת - Cross-Entropy Loss:

משמשת בבעיות **סיווג** (חיזוי קטגוריות):

סיווג בינארי:

$$\text{BCE} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)] \tag{2.12}$$

סיווג רב-מחלקתי (yportnE-ssorC lacirogetaC):

$$\text{CCE} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i) \tag{2.13}$$

כאשר C הוא מספר המחלקות.

דוגמה: סיווג סנטימנט (חיובי/שלילי) בטקסט. אם המשפט חיובי (מחלקה 1) והרשת העריכה הסתברות של 0.9, השגיאה היא: $-\log(0.9) \approx 0.105$ (שגיאה קטנה).

שגיאה ברצפים - Sequence Loss: ב-RNN, אנחנו מעבדים רצפים, כך שיש שגיאה **בכל צעד זמן**. השגיאה הכוללת היא סכום השגיאות מכל הצעדים:

$$\text{Total Loss} = \sum_{t=1}^{T} L_t \tag{2.14}$$

כאשר:

- L_t - השגיאה בצעד זמן t

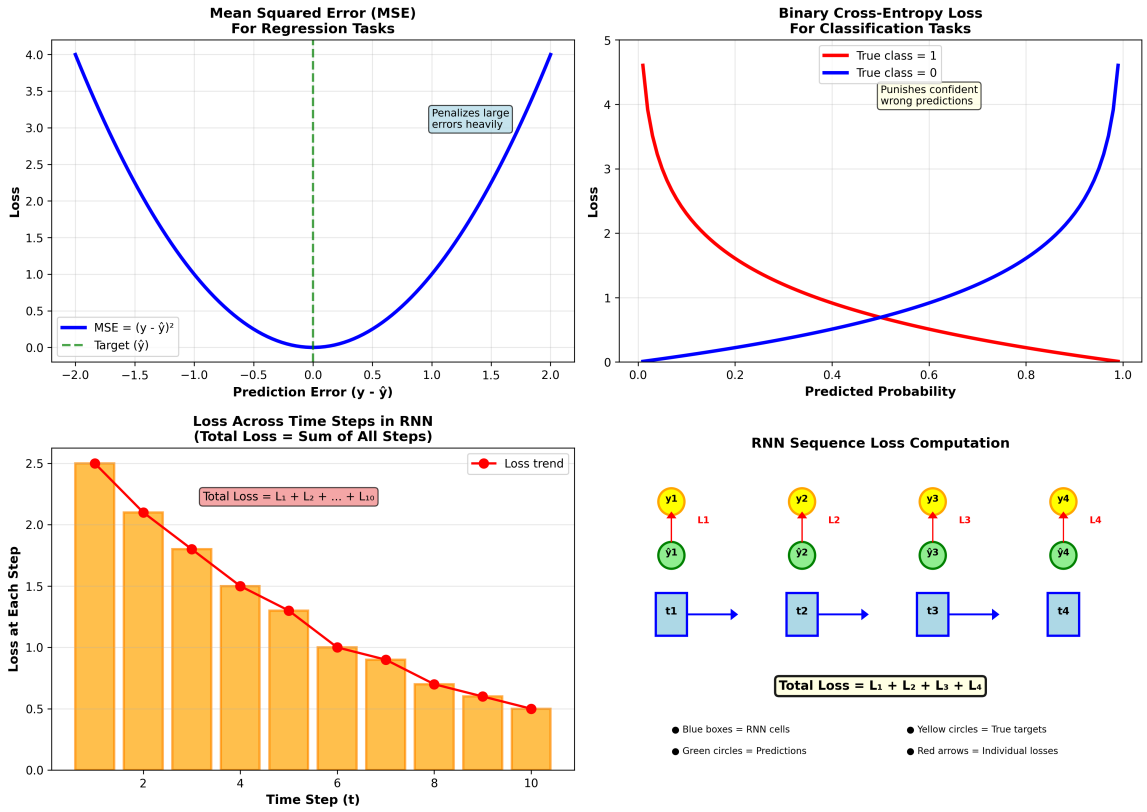
- T - אורך הרצף

דוגמה: אם אנחנו מנבאים מחירי מניות ל-5 ימים קדימה, יש 5 חיזויים ו-5 שגיאות. השגיאה הכוללת היא הסכום שלהן:

$$\text{Total Loss} = L_1 + L_2 + L_3 + L_4 + L_5 \tag{2.15}$$

איור 2.8 מציג את פונקציות השגיאה המרכזיות. למעלה משמאל: גרף MSE שמראה איך שגיאות גדולות נענשות בכוח 2. למעלה מימין: Cross-Entropy לסיווג - הקו האדום מראה שגיאה כשהמחלקה האמיתית היא 1, והכחול כשהיא 0. למטה משמאל: עמודות המראות איך השגיאה יורדת עם הזמן במהלך האימון. למטה מימין: דיאגרמה של רצף RNN - כל תא מייצר חיזוי (ŷ), משווים ליעד (y), מחשבים שגיאה (L), והשגיאה הכוללת היא הסכום.

כל הזכויות שמורות - Dr. Segal Yoram ©



איור 8.2: פונקציות שגיאה ב-RNN: Loss Functions in RNN - ארבעה דיאגרמות: MSE (משמאל למעלה) לרגרסיה, Cross-Entropy (מימין למעלה) לסיווג, שגיאה לאורך צעדי זמן (משמאל למטה), ודיאגרמת חישוב שגיאה ברצף (מימין למטה) שמראה איך השגיאה הכוללת היא סכום השגיאות מכל תא RNN.

הקשר בין פונקציות הפעלה לשגיאה: פונקציות ההפעלה ופונקציות השגיאה עובדות ביחד:

- **רגרסיה:** בדרך כלל משתמשים ב-Linear/ReLU בשכבת הפלט + MSE Loss
- **סיווג בינארי:** Sigmoid בשכבת הפלט + Binary Cross-Entropy
- **סיווג רב-מחלקתי:** Softmax בשכבת הפלט + Categorical Cross-Entropy
- **שכבות חבויות:** Tanh או ReLU (לא תלוי בשגיאה)

בעיית ה-Vanishing/Exploding Gradient: בחירת פונקציית ההפעלה משפיעה על **זרימת הגרדיאנט** במהלך האימון:

- **Sigmoid:** הנגזרת המקסימלית היא 0.25, כך שברצפים ארוכים הגרדיאנט "נעלם" (מתקרב לאפס)
- **Tanh:** הנגזרת המקסימלית היא 1, עדיין יכול להיות Vanishing אך פחות חמור
- **ReLU:** הנגזרת היא 0 או 1, פותר Vanishing אך יכול לגרום ל-Exploding

בפרקים מתקדמים נלמד על ארכיטקטורות כמו LSTM ו-GRU שפותרות בעיות אלו.

© Dr. Segal Yoram - כל הזכויות שמורות

סיכום: המבנה הבסיסי של רשת RNN פשוט להפתיע - זה אותה רשת נוירונים שאנחנו מכירים, עם לולאת משוב שמחזירה את הפלט כקלט. אבל פשטות זו מסתירה כוח עצום: היכולת לעבד סדרות בכל אורך ולזכור דפוסים לאורך זמן.

בפרק הבא נראה כיצד אפשר "לפרוס" את לולאת המשוב לאורך ציר הזמן, מה שיעזור לנו להבין בצורה ויזואלית כיצד המידע זורם דרך הרשת.

3 פריסת רשתות RNN: Unrolling RNNs

בפרק זה נלמד על טכניקת הוויזואליזציה החשובה ביותר להבנת רשתות RNN - פריסה (Unrolling). טכניקה זו הופכת את המבנה המעגלי המורכב של הרשת לשרשרת ליניארית ברורה, מה שמקל מאוד על הבנת זרימת המידע לאורך הזמן.

3.3.1 מושג הפריסה: The Unrolling Concept

הבעיה המרכזית בהבנת רשתות RNN היא שלולאת המשוב (Feedback Loop) יוצרת מבנה מעגלי שקשה לדמיין. כיצד המידע זורם? איך אתמול משפיע על היום, והיום על מחר?

הפתרון הוא טכניקה חכמה שנקראת **פריסה** או **פתיחה** (Unrolling). הרעיון הוא פשוט במיוחד:

1. **העתקת הרשת לכל צעד זמן:** עבור כל נקודת זמן בסדרה, אנחנו יוצרים עותק של הרשת. אם יש לנו 2 ימים של נתונים, נקבל 2 עותקים. אם יש 10 ימים, נקבל 10 עותקים.

2. **חיבור העותקים ברצף:** כל עותק מחובר לעותק הבא בסדר כרונולוגי. הפלט מהעותק הראשון הופך לקלט לעותק השני, ופלט השני לקלט השלישי, וכן הלאה.

3. **הפיכת הזרימה לנראית לעין:** במקום לולאה מעגלית אחת שמעבדת את כל הנתונים, אנחנו רואים שרשרת ברורה של רשתות שמעבירות מידע מימין לשמאל (או משמאל לימין, תלוי באיור).

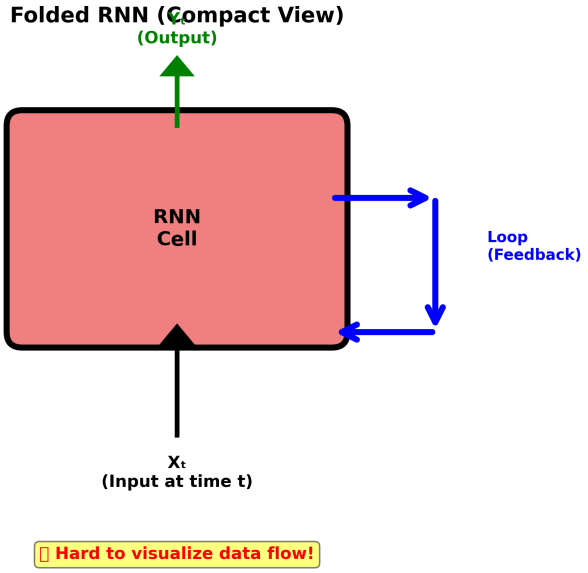
למה זה עובד? כי רשת RNN באמת עושה בדיוק את זה! בכל צעד זמן, אותה רשת מעבדת קלט חדש ומשתמשת במצב הסמוי מהצעד הקודם. הפריסה פשוט מציגה את התהליך הזה בצורה ויזואלית.

איור 3.1 מציג את המבנה המקופל של רשת RNN - תא בודד עם לולאת משוב. זה המצג הקומפקטי, אבל הלולאה המעגלית מקשה על הבנת זרימת המידע לאורך זמן. הפריסה תעזור לנו לראות את התהליך בצורה ברורה יותר.

הנוסחה הבסיסית לכל צעד זמן t היא:

$$\text{Output}_t = f(\text{Input}_t, \text{Hidden}_{t-1}) \tag{3.1}$$

כאשר:



איור 1.3: רשת RNN מקופלת: Folded RNN - המצג הקומפקטי של רשת RNN עם לולאת המשוב (החץ הכחול העגול) שמחזירה את המצב הסמוי h כקלט לצעד הבא. מבנה מעגלי זה קשה לדמיון.

- Output_t - הפלט בזמן t
- Input_t - הקלט בזמן t (למשל, מחיר המניה היום)
- Hidden_{t-1} - המצב הסמוי מהזמן הקודם (הזיכרון מאתמול)
- f - פונקציית הרשת שכוללת משקלים, הטיות ופונקציית אקטיבציה

3.3.2 דוגמת עיבוד - נתוני יומיים: Processing Example - 2-Day Data

בואו נראה דוגמה קונקרטית של חיזוי מחירי מניות. נניח שאנחנו רוצים לחזות את מחיר המניה של מחר, ויש לנו מידע על אתמול ועל היום.

השלב הראשון - אתמול (צעד זמן 1):

ביום ראשון, אנחנו מכניסים את מחיר המניה מאתמול לרשת. נניח שהמחיר היה 100 שקלים. הרשת מעבדת את הערך הזה:

- **קלט:** מחיר אתמול = 100 ש"ח
- **מצב התחלתי:** Hidden_0 (בדרך כלל מאותחל לאפסים)
- **עיבוד:** הרשת מחשבת Hidden_1 - מצב סמוי שמקודד את המידע מאתמול
- **פלט ביניים:** ערך שמשקף את ההבנה של הרשת לגבי המחיר מאתמול

השלב השני - היום (צעד זמן 2):

© Dr. Segal Yoram - כל הזכויות שמורות

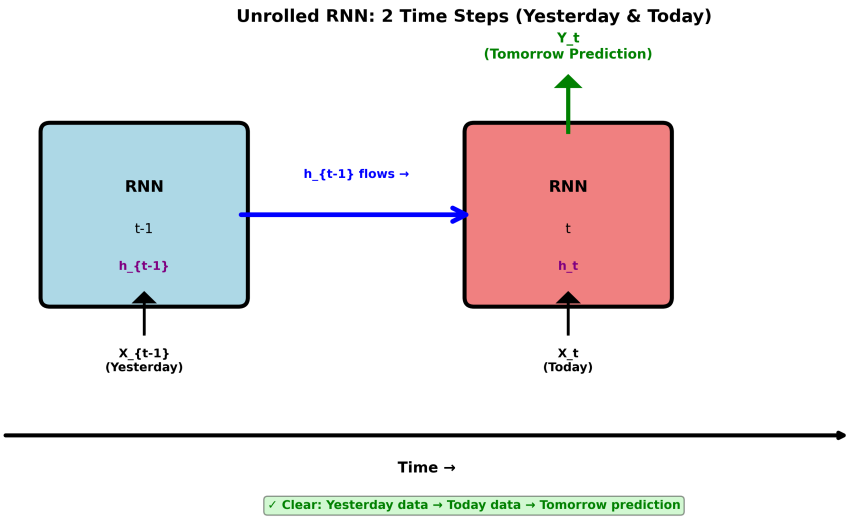
עכשיו, הרשת מקבלת את מחיר היום. נניח שהמחיר עלה ל-105 שקלים. הרשת מעבדת את הערך החדש **יחד עם המידע מאתמול**:

- **קלט:** מחיר היום = 105 ש"ח
- **זיכרון:** Hidden_1 (המצב הסמוי שחושב בשלב הקודם)
- **עיבוד:** הרשת משלבת את המידע החדש עם הזיכרון הישן
- **פלט סופי:** חיזוי למחר!

הנוסחה המדויקת לחיזוי המחיר של מחר:

$$\text{Prediction}_{\text{tomorrow}} = \text{RNN}(\text{Price}_{\text{yesterday}}, \text{Price}_{\text{today}}) \tag{3.2}$$

מה קרה כאן? הרשת למדה לזהות מגמות. היא ראתה שהמחיר עלה מ-100 ל-105 (5 שקלים, עליה של 5%). על בסיס המגמה הזו והזיכרון שלה מהאימון, היא יכולה לחזות אם המחיר ימשיך לעלות, יירד, או יישאר יציב.



איור 2.3: פריסה ל-2 צעדי זמן: Unrolled for 2 Time Steps - הרשת נפרשת לשני תאים: t=0 מעבד את מחיר אתמול (100) ו-t=1 מעבד את מחיר היום (105). המצב הסמוי h_0 זורם מהתא הראשון לשני (חץ כחול), ומאפשר לרשת לזהות את המגמה העולה.

איור 3.2 ממחיש את הפריסה לשני צעדי זמן. בצד שמאל רואים את התא הראשון (t=0) המעבד את מחיר אתמול ומייצר מצב סמוי h_0. מצב זה זורם לתא השני (t=1, בצד ימין) שמעבד את מחיר היום יחד עם הזיכרון מאתמול. החץ הכחול המקווקו מדגיש את זרימת המידע בין הצעדים - זהו לב ליבה של מנגנון הזיכרון ב-RNN.

הסדר חשוב מאוד! אנחנו חייבים להזין את הנתונים מהישן לחדש: תחילה אתמול, אחר כך היום. אם נהפוך את הסדר, החיזוי יהיה שגוי לחלוטין כי הרשת מניחה שזמן זורם קדימה.

3.3.3 הרחבה ליותר ימים: Scaling to More Days

עד כה ראינו דוגמה פשוטה עם 2 ימים. אבל מה קורה אם יש לנו 5 ימים? או 100 ימים? האם הרשת צריכה להשתנות?

התשובה המפתיעה: לא!

העיקרון נשאר בדיוק אותו הדבר:

1. **הוספת עותקים:** אם יש 5 ימים, נפרוש את הרשת ל-5 עותקים. אם יש 100 ימים, נפרוש ל-100 עותקים.
2. **שמירה על הסדר:** תמיד מהישן לחדש - היום הכי ישן נכנס ראשון, והיום הכי חדש נכנס אחרון.
3. **אותו העיקרון:** כל עותק מעביר את המצב הסמוי שלו לעותק הבא, ויוצר שרשרת של זיכרון.

דוגמה עם 4 ימים:

נניח שיש לנו מחירי מניה מ-4 ימים אחרונים: 100, 105, 103, 107 שקלים.

- **צעד 1:** קלט = 100, זיכרון התחלתי $\rightarrow$ מצב סמוי h_1
- **צעד 2:** קלט = 105, זיכרון = h_1 $\rightarrow$ מצב סמוי h_2
- **צעד 3:** קלט = 103, זיכרון = h_2 $\rightarrow$ מצב סמוי h_3
- **צעד 4:** קלט = 107, זיכרון = h_3 $\rightarrow$ **חיזוי סופי למחר**

בסוף התהליך, המצב הסמוי h_4 מכיל את כל המידע מכל 4 הימים, מסונתז ומעובד. הרשת יכולה לזהות לא רק מגמות פשוטות, אלא גם תבניות מורכבות יותר - למשל, תנודתיות גבוהה (ראינו עליה, ירידה, עליה) או מחזוריות.

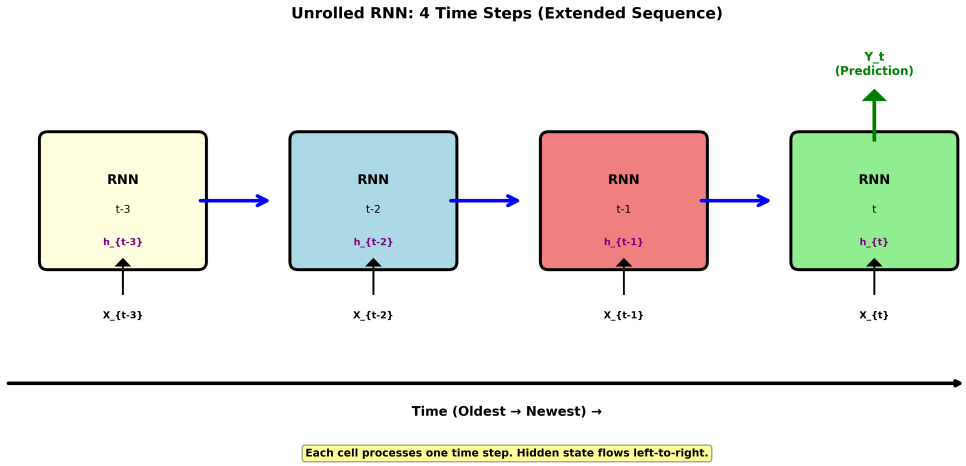
איור 3.3 מציג פריסה מלאה לארבעה צעדי זמן. ניתן לראות בבירור את שרשרת העיבוד: התא הראשון ביותר (שמאלה) מעבד 100 ומייצר h_0, שזורם לתא השני המעבד 105 ומייצר h_1, וכן הלאה. החצים הכחולים מראים את זרימת הזיכרון המצטבר. שימו לב שכל תא זהה מבחינה מבנית (שיתוף משקלים!), אך המצב הפנימי שלו שונה כי הוא מכיל זיכרון מכל הצעדים הקודמים.

כמה עותקים נדרשים? זה תלוי באורך הסדרה שלנו:

- נתוני שבוע (ימי עבודה) $\rightarrow$ 5 עותקים
- נתוני חודש (ימי מסחר) $\rightarrow$ כ-22 עותקים
- נתוני שנה $\rightarrow$ כ-252 עותקים

נקודה חשובה: למרות שיש הרבה עותקים בפריסה, זה רק לצורך ויזואליזציה! בפועל, הרשת היא **אותה רשת** שרצה 252 פעמים. המשקלים והפרמטרים זהים בכל העותקים - זה נקרא **שיתוף משקלים** (Weight Sharing), ונלמד על זה בפרק הבא.

© Dr. Segal Yoram - כל הזכויות שמורות



איור 3.3: פריסה ל-4 צעדי זמן: Unrolled for 4 Time Steps - רצף של ארבעה תאי RNN המעבדים מחירי מניות: 100, 105, 103, 107 שקלים. כל תא מעביר את המצב הסמוי שלו (h_0, h_1, h_2, h_3) לתא הבא (חצים כחולים). התא האחרון מייצר את החיזוי הסופי על בסיס כל ההיסטוריה.

מבנה הארכיטקטורה בפריסה: חשוב להבין שבכל שלב של Unrolling (פריסה בזמן), השכבות הן **שכבות מחוברות במלואן** (Fully Connected Layers). כלומר:

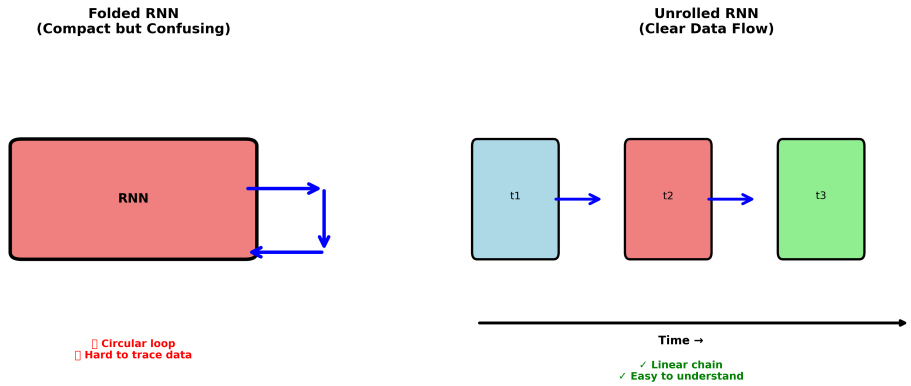
- בכל צעד זמן, שכבת הקלט מקבלת את הטוקן הבא
- השכבה החבויה מחשבת את המצב הסמוי החדש (Hidden State) לפי הקלט והמצב הקודם
- שכבת הפלט מייצרת את התוצאה לאותו צעד
- **כל הקשרים בין השכבות הם קשרים מלאים** - כל נוירון מחובר לכל נוירון בשכבה הבאה

זה אומר שבמימוש בסיסי של RNN, אין שימוש בסוגי שכבות אחרות, אלא רק שכבות מחוברות במלואן עם קשרים חוזרים.

איור 3.4 מסכם את הקונספט של פריסה. בצד שמאל רואים את המצג המקופל - תא בודד עם לולאה מעגלית שקשה להבין. בצד ימין רואים את אותה רשת בדיוק, אבל נפרשת לאורך 3 צעדי זמן. עכשיו ברור לחלוטין איך המידע זורם: מ-t=0 ל-t=1 ול-t=2. החץ הכתום הגדול מדגיש שזו לא רשת שונה - זו אותה רשת, מוצגת בשתי דרכים שונות. הפריסה היא רק כלי להבנה, לא שינוי במבנה!

סיכום הפרק:

פריסת רשתות RNN היא כלי ויזואליזציה שהופך מבנה מעגלי מורכב לשרשרת ליניארית ברורה. היא עוזרת לנו להבין כיצד המידע זורם מצעד זמן אחד למשנהו, וכיצד הרשת משתמשת בזיכרון מהעבר כדי לקבל החלטות בהווה. בפרק הבא נבין למה כל העותקים האלה בעצם חולקים את אותם המשקלים, מה שהופך את האימון ליעיל במיוחד.



איור 4.3: השוואה: מקופל לעומת פרוס: Comparison: Folded vs Unrolled - משמאל: רשת RNN מקופלת עם לולאת משוב מעגלית. מימין: אותה רשת נפרשת ל-3 צעדי זמן, והלולאה הופכת לשרשרת ליניארית ברורה. החץ הכתום מראה את הטרנספורמציה הקונספטואלית.

4 שיתוף משקלים: Weight Sharing

בפרק זה נבין את אחד העקרונות החשובים ביותר ברשתות RNN - עקרון שיתוף המשקלים. עקרון זה מבטיח שהרשת תישאר יעילה ללא קשר לאורך הרצף שעליה לעבד.

4.4.1 עקרון הפרמטרים המשותפים: Shared Parameters Principle

כאשר פורסים רשת RNN לאורך הזמן, עלול להיווצר רושם שהרשת מכילה מספר עצום של פרמטרים - משקל נפרד לכל צעד זמן. אולם במציאות, זהו רק אשליה ויזואלית. העובדה המפתיעה היא שכל העתקי הרשת המופיעים בפריסה חולקים את אותם הפרמטרים בדיוק.

נבחן את הפרמטרים השונים ברשת:

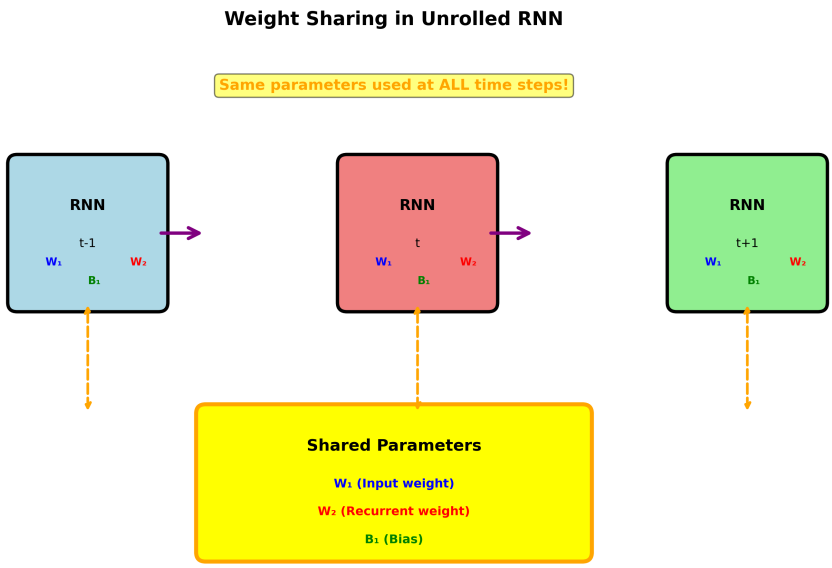
- **המשקל** W_1: אותו משקל בדיוק משמש לכל הקלטים בכל צעדי הזמן
- **הביאס** B_1: אותו ביאס בדיוק מתווסף בכל צעד
- **המשקל** W_2: למרות שנראה מועתק בפריסה, הוא זהה בכל מופע
- **כל הביאסים**: כולם משותפים לכל צעדי הזמן

נוכל להבין זאת באמצעות דוגמה פשוטה. נניח שרשת ה-RNN שלנו מכילה:

- 3 משקלים: W_1, W_2, W_h (משקל המצב הסמוי)
- 2 ביאסים: B_1, B_2

נסמן את המספר הכולל של הפרמטרים:

$$\text{ללוכה סירטמרפה רפסמ} = 3 + 2 = 5 \tag{4.1}$$



איור 1.4: שיתוף משקלים ברשת פרושה: Weight Sharing in Unrolled RNN - שלושה צעדי זמן (t-1, t, t+1) משתמשים באותם פרמטרים בדיוק: W_1 (כחול), W_2 (אדום), ו-B_1 (ירוק). החצים הכתומים המקווקוים מראים שכל התאים מחוברים לאותו מאגר פרמטרים משותף (התיבה הצהובה בתחתית).

הנקודה המרכזית: מספר זה קבוע ואינו תלוי באורך הרצף! בין אם אנו מעבדים רצף של יומיים, שבוע, חודש או שנה - תמיד יהיו לנו בדיוק 5 פרמטרים לאמן.

איור 4.1 ממחיש את קונספט שיתוף המשקלים. למרות שרואים שלושה תאי RNN שונים בצבעים שונים (כחול, אדום, ירוק), כולם מכילים את אותם הפרמטרים בדיוק! ניתן לראות בW_1, W_2, ו-B_1 מופיעים בכל תא. החצים הכתומים המקווקוים מחברים את כל התאים לתיבה הצהובה בתחתית - מאגר הפרמטרים המשותף. זה אומר שבזיכרון המחשב יש רק **קבוצה אחת** של פרמטרים, והיא משותפת לכל צעדי הזמן.

4.4.2 למה זה חשוב: Why This Matters

שיתוף המשקלים מביא לשלוש תועלות מרכזיות:

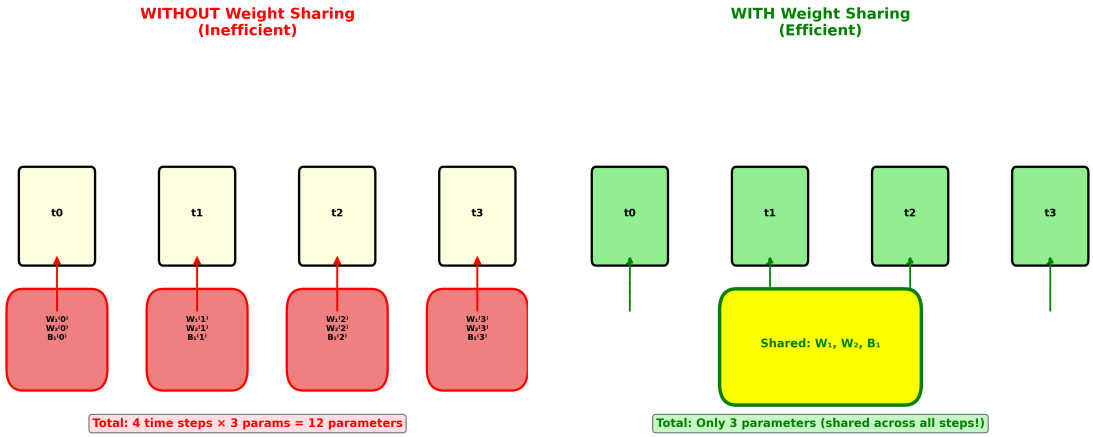
1. אין התפוצצות פרמטרים ללא שיתוף משקלים, רשת שמעבדת רצף באורך n צעדי זמן הייתה צריכה (סירטמרפה רפסם) $n \times$ פרמטרים שונים. עבור רצף של 100 צעדים עם 5 פרמטרים בסיסיים, היינו צריכים 500 פרמטרים!

עם שיתוף משקלים:

$$\text{(פצרה דרוא)}f \neq \text{סירטמרפ רפסם} \tag{4.2}$$

כלומר, מספר הפרמטרים אינו תלוי באורך הרצף כלל.

איור 4.2 מציג השוואה דרמטית. בצד שמאל רואים מה היה קורה ללא שיתוף משקלים:



איור 2.4: השוואת מספר פרמטרים: Parameter Count Comparison - משמאל: ללא שיתוף משקלים - כל צעד זמן דורש פרמטרים נפרדים (סה"כ 12 פרמטרים עבור 4 צעדים). מימין: עם שיתוף משקלים - רק 3 פרמטרים משותפים לכל הצעדים (החצים הירוקים המקווקוים).

כל תא מ-4 התאים דורש פרמטרים נפרדים משלו ($W_1^{(0)}, W_2^{(0)}, B_1^{(0)}$) עבור t_0, $W_1^{(1)}, W_2^{(1)}, B_1^{(1)}$ עבור t_1, וכו'), סך הכל 12 פרמטרים! בצד ימין רואים את הגישה היעילה: רק 3 פרמטרים משותפים (W_1, W_2, B_1) בתיבה הצהובה, וכל ארבעת התאים משתמשים בהם (החצים הירוקים). ההפרש הזה רק גדל כשהרצף מתארך!

2. **מספר קבוע לאימון** האלגוריתם שמאמן את הרשת צריך לעדכן רק את הפרמטרים המשותפים. זה אומר שהאימון יעיל הרבה יותר - אנו מעדכנים 5 ערכים במקום 500.

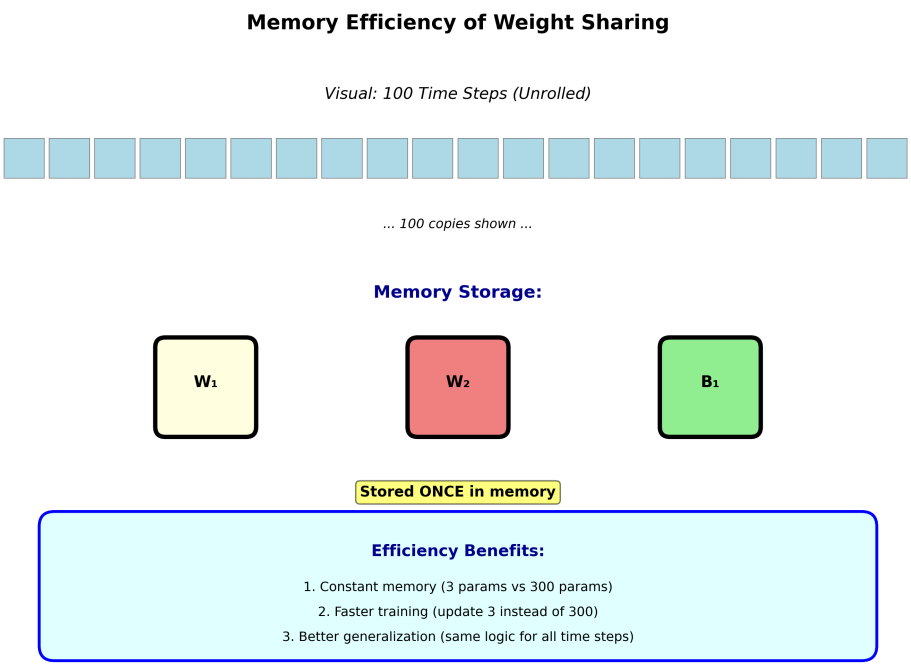
3. **למידה יעילה** כאשר הרשת לומדת כיצד לעבד קלט אחד בזמן $t = 1$, היא אוטומטית משפרת את הביצועים שלה גם בזמנים $t = 2, t = 3, \ldots$ כיוון שהיא משתמשת באותם משקלים. כל דוגמה שהרשת רואה מסייעת לשפר את הביצועים בכל צעדי הזמן.

יתרון זיכרון מבחינת זיכרון המחשב, אנו צריכים לאחסן רק את הפרמטרים המקוריים פעם אחת. בפריסה הוויזואלית אנו רואים אותם חוזרים, אך בזיכרון הם מאוחסנים פעם אחת בלבד.

איור 4.3 ממחיש את יעילות הזיכרון המדהימה של שיתוף משקלים. בחלק העליון רואים 100 תאים בפריסה ויזואלית (מיוצגים על ידי 20 תיבות כדוגמה), שעלולים ליצור רושם של מבנה עצום. אך בחלק התחתון רואים את האמת: רק 3 פרמטרים (W_1 בצהוב, W_2 באדום, B_1 בירוק) מאוחסנים בזיכרון! התיבה הכחולה בתחתית מסבירה את שלושת היתרונות: זיכרון קבוע (לא משנה כמה ארוך הרצף), אימון מהיר (מעדכנים רק 3 פרמטרים במקום 300), והכללה טובה (אותה לוגיקה לכל צעדי הזמן).

הטבלה 4.1 ממחישה בבירור את היתרון העצום של שיתוף משקלים. בעוד שללא שיתוף המשקלים מספר הפרמטרים גדל באופן לינארי עם אורך הרצף, עם שיתוף משקלים הוא נשאר קבוע.

איור 4.4 מציג גרף מרשים המשווה את שתי הגישות. ציר ה-X מייצג את אורך הרצף



איור 3.4: יעילות זיכרון: Memory Efficiency - למעלה: 100 תאי RNN בפריסה ויזואלית (מיוצגים ב-20 תיבות כחולות). למטה: רק 3 פרמטרים (W_1, W_2, B_1) מאוחסנים בזיכרון פעם אחת. היעילות: זיכרון קבוע, אימון מהיר יותר, והכללה טובה יותר.

טבלה 1.4: השוואת מספר הפרמטרים: Parameter Count Comparison

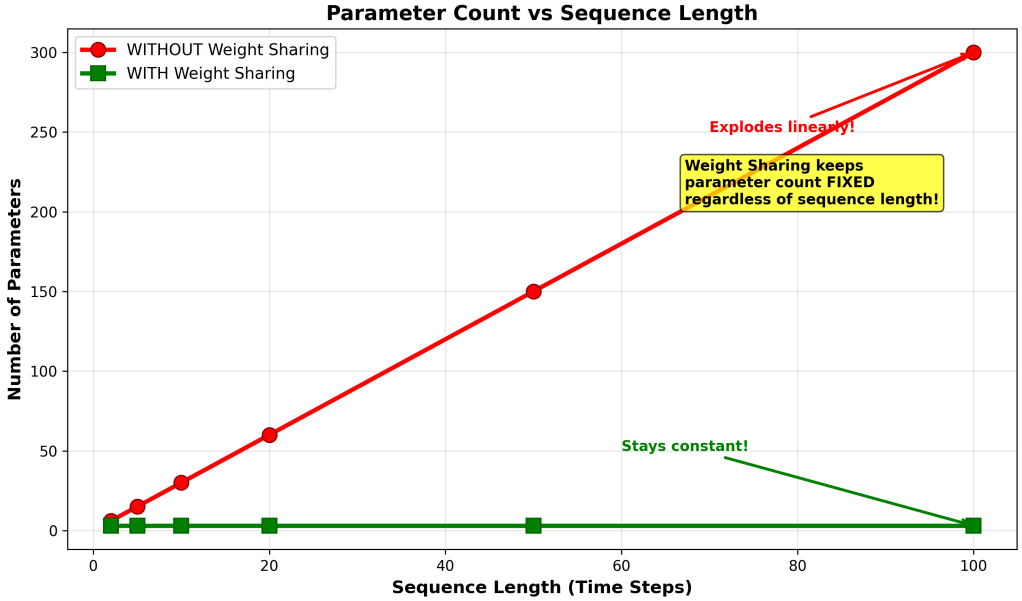
Sequence Length / אורך רצף	Without Sharing / ללא שיתוף	With Sharing / עם שיתוף
2	10	5
5	25	5
10	50	5
50	250	5
100	500	5

(מ-2 עד 100 צעדי זמן), וציר ה-Y את מספר הפרמטרים. הקו האדום עם עיגולים מראה מה קורה ללא שיתוף משקלים - מספר הפרמטרים מתפוצץ באופן לינארי ומגיע ל-300 פרמטרים עבור רצף של 100 צעדים! לעומת זאת, הקו הירוק עם ריבועים מראה את הקסם של שיתוף משקלים - מספר הפרמטרים נשאר שטוח לחלוטין ב-3, ללא קשר לאורך הרצף. התיבה הצהובה מדגישה: "שיתוף משקלים שומר על מספר הפרמטרים קבוע ללא קשר לאורך הרצף!"

דוגמה מעשית נחזור לדוגמה שלנו מהפרקים הקודמים - ניבוי מחירי מניות. נניח שאנו רוצים לנבא את מחיר המניה מחר על סמך המחירים ב-30 הימים האחרונים.

ללא שיתוף משקלים:

- לכל אחד מ-30 הימים יש משקלים וביאסים נפרדים



איור 4.4: גרף התרחבות: Scaling Graph - הקו האדום מראה התפוצצות לינארית של מספר הפרמטרים ללא שיתוף משקלים (עד 300 פרמטרים עבור רצף באורך 100). הקו הירוק מראה שעם שיתוף משקלים, מספר הפרמטרים נשאר קבוע ב-3 ללא קשר לאורך הרצף!

- אם לכל יום יש 5 פרמטרים, סה"כ $30 \times 5 = 150$ פרמטרים
- האימון יהיה איטי ויצריך הרבה נתונים

עם שיתוף משקלים:

- כל 30 הימים משתמשים באותם 5 פרמטרים
- הרשת לומדת "כיצד לעבד יום" ומיישמת את הידע הזה על כל הימים
- האימון מהיר יותר ויעיל יותר

הנוסחה הכללית אם נסמן:

- P = מספר הפרמטרים הבסיסיים ברשת RNN
- T = אורך הרצף (מספר צעדי הזמן)

אזי:

$$\text{לעופב סירטמרפ רפסמ} = P \quad (\text{ב יולת יתלב-}T) \tag{4.3}$$

בעוד שאילולא שיתוף משקלים היינו מקבלים:

$$\text{ויותיש אלל סירטמרפ רפסמ} = P \times T \tag{4.4}$$

זוהי אחת הסיבות המרכזיות להצלחתן של רשתות RNN בעיבוד רצפים. הן מצליחות לתפוס תבניות זמניות מורכבות תוך שמירה על מספר פרמטרים סביר, מה שמאפשר למידה יעילה ויישום מעשי.

סיכום שיתוף המשקלים ברשתות RNN הוא לא רק טריק טכני - זהו עקרון יסוד המאפשר לרשתות אלו לעבוד. הוא מבטיח:

- מודל קומפקטי עם מספר פרמטרים קבוע
- אימון יעיל שלא גדל עם אורך הרצף
- שימוש חכם בזיכרון
- יכולת הכללה - מה שנלמד על קלט אחד מיושם על כולם

בפרק הבא נראה שלשיתוף המשקלים יש גם צד אפל - כאשר מעבדים רצפים ארוכים, הוא יכול להוביל לבעיות בתהליך האימון שנקראות "בעיות הגרדיאנט".

5 בעיית הגרדיאנט: The Gradient Problem

עד כה למדנו כיצד רשתות RNN עובדות, איך פורסים אותן לאורך זמן, וכיצד שיתוף המשקלים חוסך זיכרון ומאפשר אימון יעיל. אולם, כאשר מנסים לאמן רשתות RNN על סדרות ארוכות, מתעוררת בעיה חמורה שמקשה מאוד על תהליך הלמידה. פרק זה יסביר את אחת הבעיות המרכזיות באימון רשתות RNN: **בעיית הגרדיאנט**.

בתהליך האימון של רשתות נוירונים, אנו משתמשים באלגוריתם **התפשטות לאחור** (Back-propagation) כדי לחשב את הגרדיאנט - השיפוע שמכוון אותנו כיצד לשנות את המשקלים. כאשר הרשת פרוסה לאורך 50 או 100 צעדי זמן, הגרדיאנט צריך "לזרום" אחורה דרך כל הצעדים הללו. כאן מתחילה הבעיה.

5.5.1 גרדיאנטים מתפוצצים: Exploding Gradients

מהי הבעיה?

נזכור שברשת RNN המשקל W_2 מחבר את הפלט Y_t של צעד זמן אחד לקלט של הצעד הבא. כאשר מבצעים התפשטות לאחור דרך n צעדי זמן, הגרדיאנט מוכפל ב-W_2 שוב ושוב. אם $W_2 > 1$, כל הכפלה מגדילה את הגרדיאנט.

החישוב המתמטי:

כאשר $W_2 = 2$ (גדול מ-1), וסדרת הזמן מכילה 50 צעדים:

$$\text{Gradient} \propto W_2^n = 2^{50} \tag{5.1}$$

כמה זה 2^{50}? זהו מספר ענק:

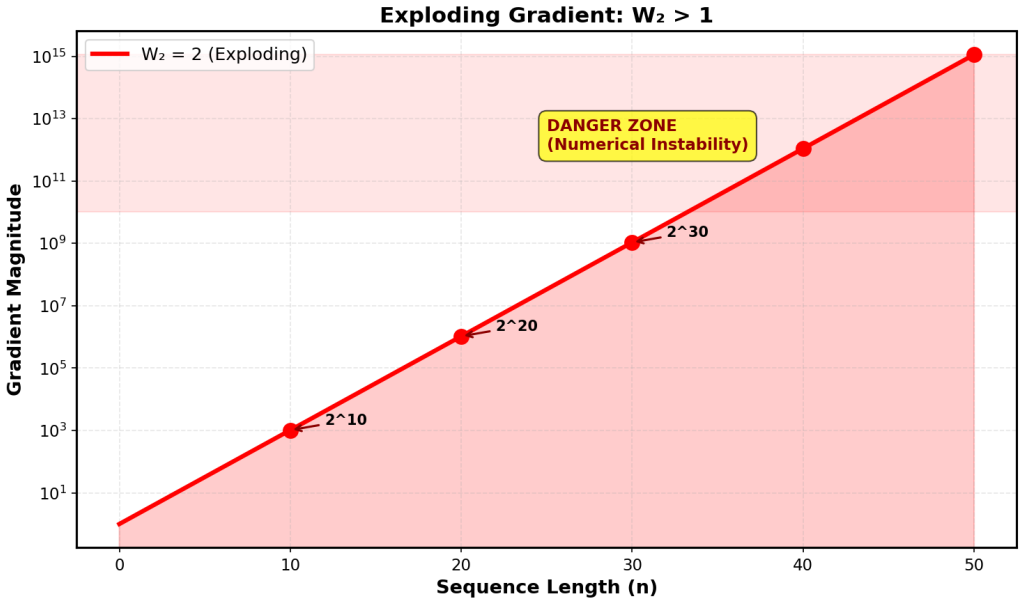
$$2^{50} = 1{,}125{,}899{,}906{,}842{,}624 \approx 1.13 \times 10^{15} \tag{5.2}$$

זהו מספר עם 16 ספרות! גרדיאנט כל כך גדול יגרום לצעדים ענקיים באופטימיזציה.

מה קורה בתהליך האימון?

כאשר הגרדיאנט "מתפוצץ" לערכים אסטרונומיים:

- **צעדים ענקיים באופטימיזציה**: המשקלים משתנים בקפיצות גדולות מדי
- **אי-יציבות נומרית**: המספרים עלולים לחרוג ממה שהמחשב יכול לייצג (NaN - Not a Number)
- **אי-התכנסות**: האלגוריתם "קופץ" מסביב למינימום במקום להתקרב אליו
- **אובדן שליטה**: אי אפשר לכוון את תהליך הלמידה



איור 1.5: התנהגות גרדיאנט מתפוצץ: Exploding Gradient Behavior - כאשר $W_2 = 2$, הגרדיאנט גדל באופן אקספוננציאלי ככל שאורך הסדרה גדל. הגרף מציג עקומת צמיחה אקספוננציאלית שמגיעה לערכים אסטרונומיים.

איור 5.1 ממחיש את הבעיה: ככל שאורך הסדרה גדל (ציר ה-x), גודל הגרדיאנט (ציר ה-y) גדל באופן דרמטי. הגרף מציג עקומה אקספוננציאלית המתנשאת כלפי מעלה.

דוגמה מספרית:

- בצעד 10: $2^{10} = 1{,}024$ (אלף)
- בצעד 20: $2^{20} = 1{,}048{,}576$ (מיליון)
- בצעד 30: $2^{30} = 1{,}073{,}741{,}824$ (מיליארד)
- בצעד 50: $2^{50} \approx 1.13 \times 10^{15}$ (קוודרילין!)

5.5.2 גרדיאנטים מתכווצים: Vanishing Gradients

הבעיה ההפוכה

אם $W_2 < 1$, קורה בדיוק ההפך. כאשר מכפילים מספר קטן מ-1 בעצמו פעמים רבות, התוצאה מתכווצת ושואפת לאפס.

החישוב המתמטי:

כאשר $W_2 = 0.5$ (קטן מ-1), וסדרת הזמן מכילה 50 צעדים:

$$\text{Gradient} \propto W_2^n = 0.5^{50} \tag{5.3}$$

כמה זה 0.5^{50}? זהו מספר זעיר:

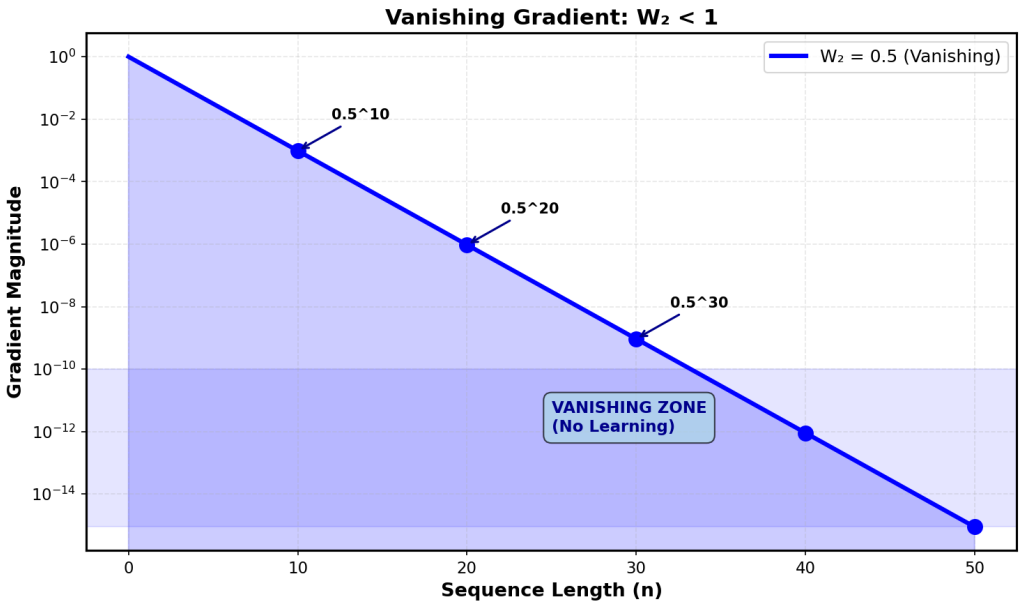
$$0.5^{50} = \frac{1}{2^{50}} \approx 8.88 \times 10^{-16} \tag{5.4}$$

זהו מספר עם 15 אפסים אחרי הנקודה העשרונית! גרדיאנט כל כך קטן כמעט ולא משנה את המשקלים.

מה קורה בתהליך האימון?

כאשר הגרדיאנט "מתכווץ" לערכים אינפיניטסימליים:

- **צעדים זעירים באופטימיזציה**: המשקלים כמעט לא משתנים
- **אובדן זיכרון**: המידע מצעדי זמן מוקדמים "נשכח" לחלוטין
- **אי-הגעה לאופטימום**: האלגוריתם תקוע רחוק מהמינימום
- **למידה איטית מאוד**: עשוי לקחת זמן אינסופי להתכנס



איור 2.5: התנהגות גרדיאנט מתכווץ: Vanishing Gradient Behavior - כאשר $W_2 = 0.5$, הגרדיאנט קורס באופן אקספוננציאלי ככל שאורך הסדרה גדל. הגרף מציג עקומת ריקבון אקספוננציאלית שמתקרבת במהירות לאפס.

איור 5.2 ממחיש את הריקבון האקספוננציאלי: ככל שאורך הסדרה גדל, גודל הגרדיאנט מתכווץ במהירות לכיוון אפס.

דוגמה מספרית:

כל הזכויות שמורות - Dr. Segal Yoram ©

- בצעד 10: $0.5^{10} \approx 0.001$ (אלפית)
- בצעד 20: $0.5^{20} \approx 9.5 \times 10^{-7}$ (פחות ממיליונית)
- בצעד 30: $0.5^{30} \approx 9.3 \times 10^{-10}$ (פחות ממיליארדית)
- בצעד 50: $0.5^{50} \approx 8.9 \times 10^{-16}$ (כמעט אפס!)

5.5.3 הקשר להתפשטות לאחור: Connection to Backpropagation

מדוע זה קורה?

בתהליך ההתפשטות לאחור, אנו משתמשים ב**כלל השרשרת** (Chain Rule) של חשבון דיפרנציאלי. כדי לחשב את הגרדיאנט של המשקל W_1 ביחס לפונקציית ההפסד L, עלינו להכפיל את הגרדיאנטים לאורך כל הרשת הפרוסה:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial y_{50}} \cdot \frac{\partial y_{50}}{\partial y_{49}} \cdot \frac{\partial y_{49}}{\partial y_{48}} \cdots \frac{\partial y_2}{\partial y_1} \cdot \frac{\partial y_1}{\partial W_1} \tag{5.5}$$

בכל שלב, הנגזרת $\frac{\partial y_{t+1}}{\partial y_t}$ כוללת את המשקל W_2. לכן:

$$\frac{\partial L}{\partial W_1} \propto W_2^n \tag{5.6}$$

כאשר n הוא מספר צעדי הזמן. זו בדיוק הסיבה שהגרדיאנט מתפוצץ כאשר $W_2 > 1$ או מתכווץ כאשר $W_2 < 1$.

השלכות מעשיות:

- **קושי באימון סדרות ארוכות**: ככל שהסדרה ארוכה יותר, הבעיה מחמירה
- **אובדן תלות ארוכת טווח**: הרשת לא יכולה ללמוד קשרים בין אירועים מרוחקים בזמן
- **צורך בפתרון**: רשתות RNN סטנדרטיות אינן מתאימות ליישומים רבים

5.5.4 תצוגה מקדימה של הפתרון: רשתות LSTM: Solution Preview: LSTM Networks

איך פותרים את בעיית הגרדיאנט?

למרבה המזל, חוקרים פיתחו פתרון אלגנטי לבעיה זו: רשתות LSTM (Long Short-Term Memory - זיכרון ארוך-קצר טווח). רשתות אלו הן סוג מתקדם של RNN שנועד במיוחד לפתור את בעיות הגרדיאנט.

העיקרון המרכזי: שערים

רשתות LSTM מוסיפות מבנה מיוחד הנקרא **שערים** (Gates). שערים אלו שולטים בזרימת המידע דרך הרשת:

- **שער השכחה** (Forget Gate): מחליט איזה מידע מהעבר לזרוק

© Dr. Segal Yoram - כל הזכויות שמורות

- **שער הקלט** (Input Gate): מחליט איזה מידע חדש לאחסן
- **שער הפלט** (Output Gate): מחליט איזה מידע להעביר הלאה

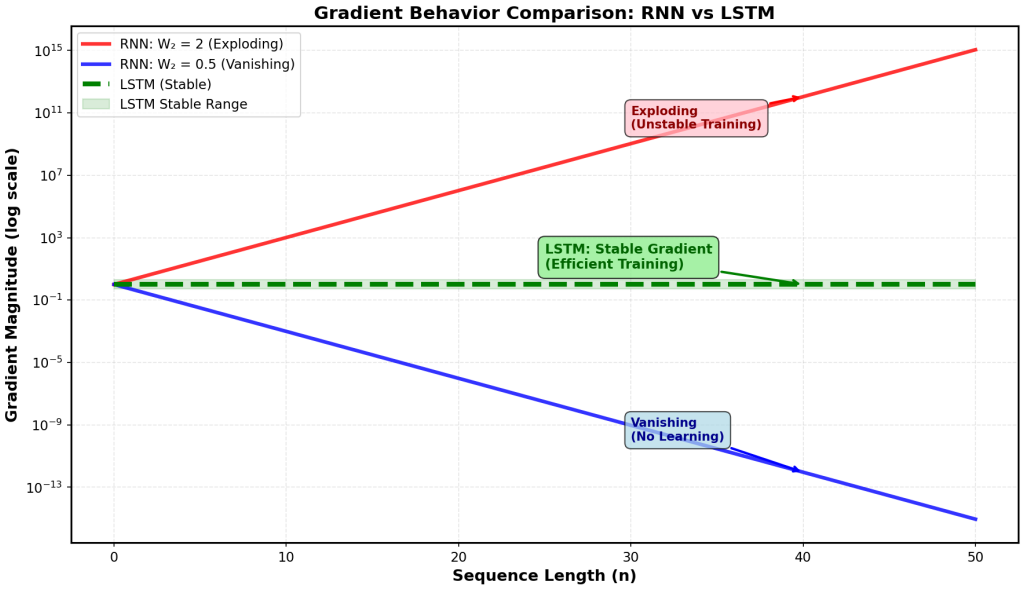
מדוע זה עוזר?

השערים יוצרים "כביש מהיר" למידע לזרום דרך הרשת מבלי להתפוצץ או להתכווץ. במקום להכפיל ב-W_2 שוב ושוב, המידע יכול לעבור ישירות דרך הזמן עם שינויים מינימליים.

$$\text{LSTM Gradient} \approx \text{constant (not exponential!)} \tag{5.7}$$

יתרונות LSTM:

- **גרדיאנטים יציבים**: השערים מונעים התפוצצות והתכווצות
- **למידת תלות ארוכת טווח**: יכולת לזכור מידע מצעדים רחוקים
- **אימון יעיל יותר**: התכנסות מהירה יותר לפתרון טוב
- **שימוש נרחב**: הסטנדרט בתעשייה ליישומי RNN



איור 3.5: השוואת התנהגות גרדיאנטים: Gradient Behavior Comparison - השוואה בין גרדיאנט ב-RNN רגיל (קווים אדום וכחול מציגים התפוצצות והתכווצות) לבין גרדיאנט ב-LSTM (קו ירוק יציב). ניכר שה-LSTM שומר על גרדיאנט יציב גם בסדרות ארוכות.

איור 5.3 מציג את ההבדל הדרמטי בין רשת RNN רגילה לבין LSTM. בעוד שהגרדיאנט ב-RNN רגיל מתפוצץ או מתכווץ, הגרדיאנט ב-LSTM נשאר יציב.

מבט קדימה

רשתות LSTM הן נושא מרתק ומורכב המצדיק ספר שלם בפני עצמו. בפרק זה הכרנו את הבעיה שהן פותרות ואת העיקרון הבסיסי שלהן. בספרות המומלצת בסוף הספר תמצאו מקורות ללימוד מעמיק של LSTM.

סיכום הפרק

- רשתות RNN סובלות מבעיית גרדיאנטים כאשר אורך הסדרה גדל
- **גרדיאנטים מתפוצצים** ($W_2 > 1$): הגרדיאנט גדל אקספוננציאלית ל-W_2^n
- **גרדיאנטים מתכווצים** ($W_2 < 1$): הגרדיאנט קורס אקספוננציאלית ל-$W_2^n \to 0$
- שתי הבעיות מונעות אימון יעיל של רשתות RNN על סדרות ארוכות
- רשתות LSTM פותרות את הבעיה באמצעות מנגנון שערים
- LSTM הוא הסטנדרט המקובל ליישומי RNN בעולם האמיתי

נקודות חשובות לזכור:

טבלה 1.5: סיכום בעיות הגרדיאנט: Gradient Problems Summary

Phenomenon / תופעה	**Condition / תנאי**	**Result / תוצאה**	**Impact / השפעה**
Exploding Gradient / גרדיאנט מתפוצץ	$W_2 > 1$	$W_2^n \to \infty$	אי-יציבות
Vanishing Gradient / גרדיאנט מתכווץ	$W_2 < 1$	$W_2^n \to 0$	למידה איטית
LSTM	שערים	יציבות	אימון יעיל

בפרק זה סיימנו את היסודות של רשתות RNN. עכשיו אתם מכירים את המוטיבציה (**פרק 1**), את המבנה הבסיסי (**פרק 2**), טכניקת הפריסה (**פרק 3**), עקרון שיתוף המשקלים (**פרק 4**), ואת האתגרים באימון (**פרק 5**). ידע זה מהווה בסיס מוצק להמשך לימוד ארכיטקטורות מתקדמות כמו LSTM ו-GRU.

© Dr. Segal Yoram - כל הזכויות שמורות